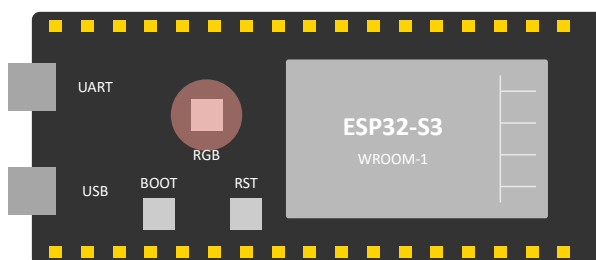


ESP32-S3

Базовый курс в десяти уроках
со сквозным проектом «умная лампа»



0. Знакомство с чипом и настройка среды
1. Цифровой выход: светодиод
2. Последовательный порт: разговор с компьютером
3. Цифровой вход: кнопки и прерывания
4. ШИМ (LEDC): плавная яркость
5. АЦП: читаем аналоговые сигналы
6. Шина I²C: дисплей
7. FreeRTOS: задачи и синхронизация
8. Wi-Fi и веб-сервер
9. Сон и энергосбережение

Все примеры — для Arduino-ESP32 версии 3.x (на базе ESP-IDF 5.x)
и отладочной платы ESP32-S3-DevKitC-1 (или совместимой).

Содержание

0	Знакомство с ESP32-S3 и настройка среды	3
0.1	Что такое ESP32-S3	3
0.2	Сравнение с «родственниками»	4
0.3	Модули и маркировка	4
0.4	Отладочная плата ESP32-S3-DevKitC-1	5
0.5	Установка среды	5
0.5.1	Если прошивка не начинается	6
0.6	Структура Arduino-скетча	6
0.7	Сквозной проект: умная лампа	7
1	Цифровой выход: светодиод	8
1.1	Как устроен выход GPIO	8
1.2	Схема подключения	9
1.3	Классический Blink	9
1.4	Мигание без delay()	10
1.5	Встроенный RGB-светодиод	10
1.6	Шаг проекта 1: светодиод лампы и «пульс»	11
2	Последовательный порт: разговор с компьютером	12
2.1	Кадр UART	12
2.2	Serial и Serial Monitor	12
2.3	Шаг проекта 2: команды с компьютера	13
2.4	Второй UART: связь с другими устройствами	14
3	Цифровой вход: кнопки и прерывания	15
3.1	Проблема «висящего» входа	15
3.2	Опрос кнопки	16
3.3	Дребезг контактов	16
3.4	Шаг проекта 3: кнопка-выключатель	17
3.5	Прерывания	18
3.6	Strapping-выводы	19
4	ШИМ (LEDC): плавная яркость	20
4.1	Что такое ШИМ	20
4.2	Периферия LEDC	20
4.3	Яркость и восприятие: гамма-коррекция	21
4.4	Шаг проекта 4: яркость лампы	21
4.5	Другое применение ШИМ: сервопривод	22
5	АЦП: читаем аналоговые сигналы	24
5.1	Как работает АЦП	24
5.1.1	Особенности АЦП ESP32-S3	24
5.2	Потенциометр	25
5.3	Шаг проекта 5: ручка яркости	25
5.4	Измерение напряжения выше 3,3 В	26

6	Шина I²C: дисплей	28
6.1	Топология шины	28
6.2	Временная диаграмма	29
6.3	Сканер шины	29
6.4	Первый вывод на экран	29
6.5	Шаг проекта 6: экран состояния	30
7	FreeRTOS: задачи и синхронизация	32
7.1	Зачем нужна операционная система реального времени	32
7.2	Первые задачи	33
7.3	Общие данные: очередь и мьютекс	33
7.4	Шаг проекта 7: лампа на задачах	34
8	Wi-Fi и веб-сервер	37
8.1	Режимы Wi-Fi	37
8.2	Подключение к сети	37
8.3	Как работает HTTP-запрос	38
8.4	Шаг проекта 8: управление через браузер	38
8.5	Если роутера нет: собственная точка доступа	39
9	Сон и энергосбережение	41
9.1	Режимы питания	41
9.2	Сколько проработает устройство от батареи	41
9.3	Deep-sleep по таймеру	42
9.4	Пробуждение кнопкой	43
9.5	Шаг проекта 9: лампа засыпает	43
A	Полный код умной лампы	45
B	Справочник	49

Урок 0. Знакомство с ESP32-S3 и настройка среды

Цель	понять, из чего состоит ESP32-S3, как устроена отладочная плата, установить среду и залить первую программу
Опирается на	ничего — это стартовый урок
Шаг проекта	знакомимся с проектом «умная лампа», который будем собирать весь курс
Понадобится	плата ESP32-S3-DevKitC-1, USB-кабель с передачей данных

0.1 Что такое ESP32-S3

ESP32-S3 — система на кристалле (SoC) компании Espressif. Главное:

- **Два ядра Xtensa LX7** до 240 MHz с векторными инструкциями (ускорение DSP и нейросетей).
- **512 KB SRAM** на кристалле, 384 KB ROM, до 16 MB внешней Flash и до 16 MB внешней PSRAM (в том числе в режиме Octal SPI).
- **Wi-Fi 802.11 b/g/n** (2,4 GHz) и **Bluetooth 5 LE**. Классического Bluetooth (BR/EDR) *нет* — в отличие от первого ESP32.
- **Нативный USB OTG** и встроенный **USB-Serial/JTAG**: прошивать и отлаживать можно без отдельного преобразователя.
- **45 GPIO**, 2 АЦП по 12 бит (20 каналов), 14 сенсорных (touch) входов, 3 UART, 2 I²C, 2 I²S, 4 SPI (два доступны пользователю), LEDC, MCPWM, RMT, TWAI (CAN), интерфейс камеры и LCD.
- **Сопроцессоры ULP** (RISC-V и FSM), которые работают, пока основные ядра спят.

Не нужно запоминать всё сразу: каждый блок схемы ниже мы по очереди задействуем в уроках курса.

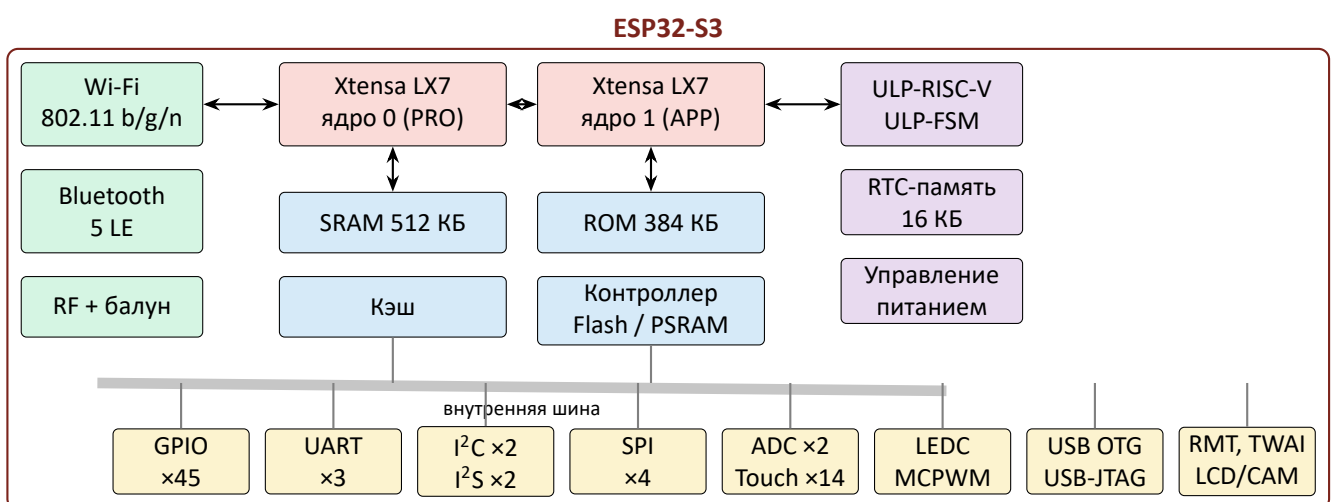


Рис. 1: Упрощённая блок-схема ESP32-S3

0.2 Сравнение с «родственниками»

	ESP32	ESP32-S3	ESP32-C3
Ядро	2× Xtensa LX6, 240 МГц	2× Xtensa LX7, 240 МГц	1× RISC-V, 160 МГц
SRAM	520 КБ	512 КБ	400 КБ
Bluetooth	Classic + BLE 4.2	BLE 5	BLE 5
USB	нет	OTG + Serial/JTAG	Serial/JTAG
GPIO	34	45	22
Особенности	ЦАП, Ethernet MAC	векторные инструкции, камера, LCD	дешёвый, малый

Таблица 1: Сравнение популярных чипов Espressif

0.3 Модули и маркировка

Чип почти всегда используется в составе *модуля* (ESP32-S3-WROOM-1, WROOM-1U, MINI-1), где уже есть Flash, кварц и антенна. Суффикс модуля кодирует объём памяти:

ESP32-S3-WROOM-1-N16R8

Flash 16 МБ PSRAM 8 МБ (Octal)

Осторожно

В модулях с **Octal PSRAM** (R8, R16V) выводы **GPIO35, GPIO36, GPIO37** заняты памятью — не используйте их. Выводы **GPIO26–GPIO32** всегда заняты SPI-Flash и на плату обычно не выведены.

0.4 Отладочная плата ESP32-S3-DevKitC-1

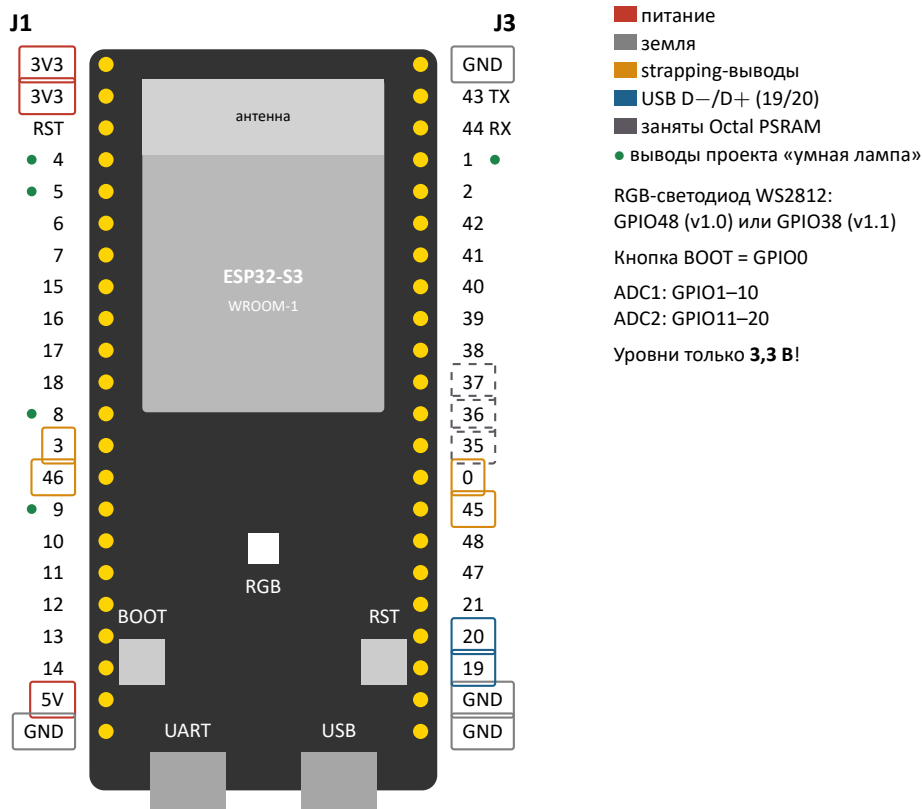


Рис. 2: Разводка выводов ESP32-S3-DevKitC-1 (вид сверху). Числа — номера GPIO. Сверьтесь с документацией именно вашей ревизии платы.

У платы **два USB-разъёма:**

- **UART** — через микросхему-преобразователь (CP2102N/CH343) на UART0 (**GPIO43/GPIO44**). Самый надёжный вариант для прошивки и вывода сообщений.
- **USB** — напрямую к выводам нативного USB чипа (**GPIO19/GPIO20**). Позволяет прошивать, отлаживать через JTAG и превращать плату в USB-клавиатуру, MIDI-устройство и т. п.

0.5 Установка среды

Для уроков используем Arduino IDE 2.x (подойдёт и PlatformIO — код не меняется).

1. Установите Arduino IDE 2 с сайта <https://www.arduino.cc/en/software>.
2. File → Preferences → Additional boards manager URLs, добавьте:
https://espressif.github.io/arduino-esp32/package_esp32_index.json
3. Tools → Board → Boards Manager, найдите **esp32 by Espressif Systems**, установите версию 3.x.
4. Выберите плату **ESP32S3 Dev Module** и порт.
5. Если подключены через разъём **USB** (нативный), включите Tools → USB CDC On Boot: Enabled, иначе сообщения от платы не дойдут до компьютера.
6. Для модулей с PSRAM выберите Tools → PSRAM: OPI PSRAM (для R8) или QSPI PSRAM (для R2).

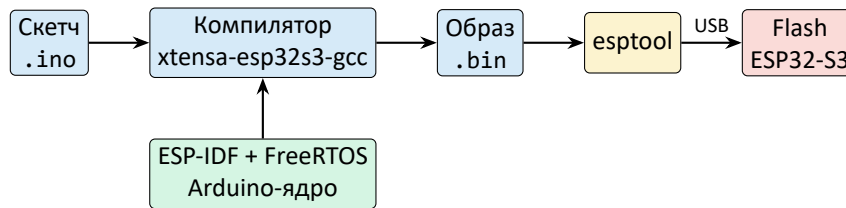
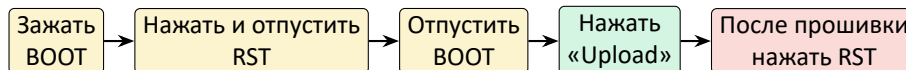


Рис. 3: Путь от исходного кода до прошивки

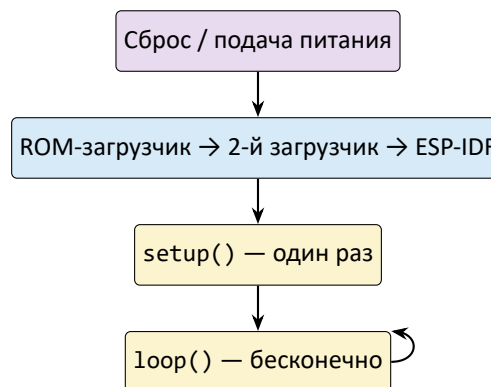
Если прошивка не начинается

Чип входит в режим загрузчика, если при сбросе **GPIO0** притянут к земле. Обычно IDE делает это автоматически, но если видите Failed to connect:



0.6 Структура Arduino-скетча

Любой скетч состоит из двух функций: `setup()` выполняется один раз после включения, `loop()` — повторяется бесконечно. (На самом деле под ними работает операционная система FreeRTOS; она нам понадобится только в уроке 7.)



Первая программа ничего не подключает: она только сообщает о себе через USB. Как именно работает этот канал связи, разберём в уроке 2, а пока просто проверим, что всё настроено.

Листинг 1: Первый скетч: «Hello, ESP32-S3»

```

void setup() {
  Serial.begin(115200);           // открыть канал связи с компьютером
  delay(500);                     // дать USB время подняться
  Serial.println("Привет, ESP32-S3!");
  Serial.printf("Частота CPU: %lu МГц\n", getCpuFrequencyMhz());
  Serial.printf("Свободно SRAM: %lu байт\n", ESP.getFreeHeap());
  Serial.printf("PSRAM: %lu байт\n", ESP.getPsramSize());
  Serial.printf("Flash: %lu байт\n", ESP.getFlashChipSize());
}

void loop() {
  Serial.printf("Прошло %lu мс\n", millis());
  delay(1000);
}
  
```

Откройте Tools → Serial Monitor, установите скорость 115200 — должны появиться сообщения.

0.7 Сквозной проект: умная лампа

Чтобы уроки не были набором разрозненных примеров, все они работают на один проект. К концу курса у нас будет настольная лампа, которая:

- включается кнопкой и командой с компьютера;
- плавно меняет яркость ручкой-потенциометром;
- показывает своё состояние на OLED-экране;
- управляется с телефона через браузер;
- засыпает, когда выключена, и просыпается от кнопки.

Каждый урок добавляет ровно одну возможность, опираясь на уже изученное. Выводы платы закреплены за деталями с самого начала и дальше не меняются:

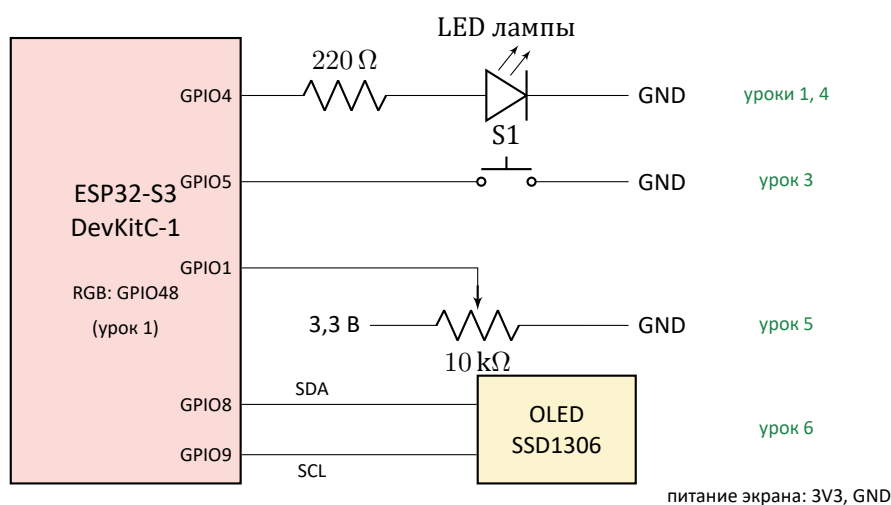


Рис. 4: Итоговая схема умной лампы. Детали добавляются по мере прохождения уроков

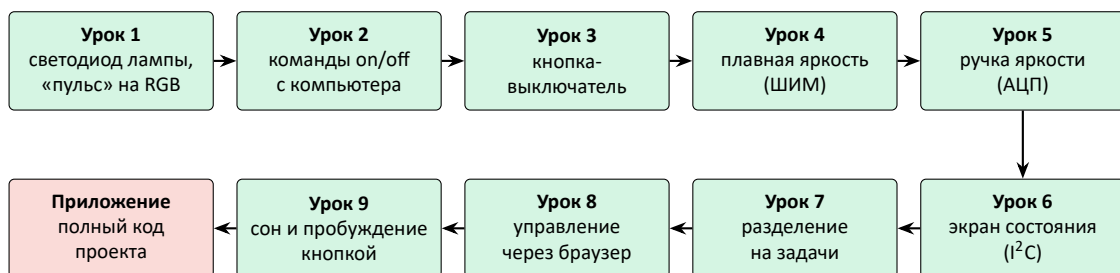


Рис. 5: Как растёт проект от урока к уроку

Итоги урока

- ESP32-S3 — двухъядерный чип с Wi-Fi, BLE, USB и богатой периферией; работает на 3,3 В.
- Прошивка заливается из Arduino IDE; при проблемах помогает ручной вход в загрузчик (BOOT + RST).
- Скетч — это `setup()` (один раз) и `loop()` (бесконечно).

Задания для самостоятельной работы

1. Выясните, какой модуль стоит на вашей плате (надпись на экране модуля) и сколько у него Flash и PSRAM. Сверьте с выводом скетча.
2. Выведите MAC-адрес чипа: `ESP.getEfuseMac()`.
3. Измените частоту CPU функцией `setCpuFrequencyMhz(80)` и проверьте результат.

Урок 1. Цифровой выход: светодиод

Цель	научиться управлять выводом GPIO как выходом, правильно подключать светодиод, отмерять время без остановки программы
Опирается на	урок 0: установка среды, <code>setup()</code> и <code>loop()</code>
Шаг проекта	подключаем светодиод лампы и «пульс» — индикатор того, что плата работает
Понадобится	макетная плата, светодиод, резистор 220 – 330 Ω , провода

1.1 Как устроен выход GPIO

Выход GPIO — это пара транзисторов (двухтактная схема, push-pull). Логическая 1 подключает вывод к 3,3 V, логический 0 — к земле.

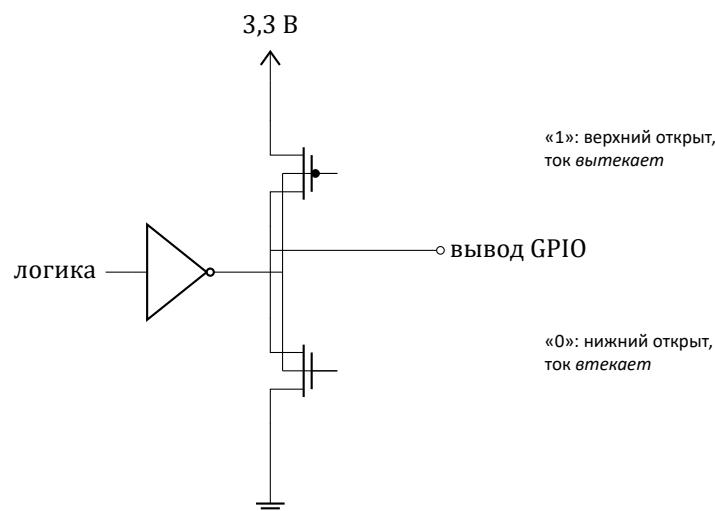


Рис. 6: Упрощенная схема выходного каскада GPIO

Осторожно

- ESP32-S3 работает с логикой **3,3 В**. Подача 5 V на вывод может вывести чип из строя.
- Допустимый ток вывода — до ≈ 40 mA (по умолчанию сила выхода ограничена ≈ 20 mA). Моторы, реле и мощные светодиоды подключаются только через транзистор или драйвер.

1.2 Схема подключения

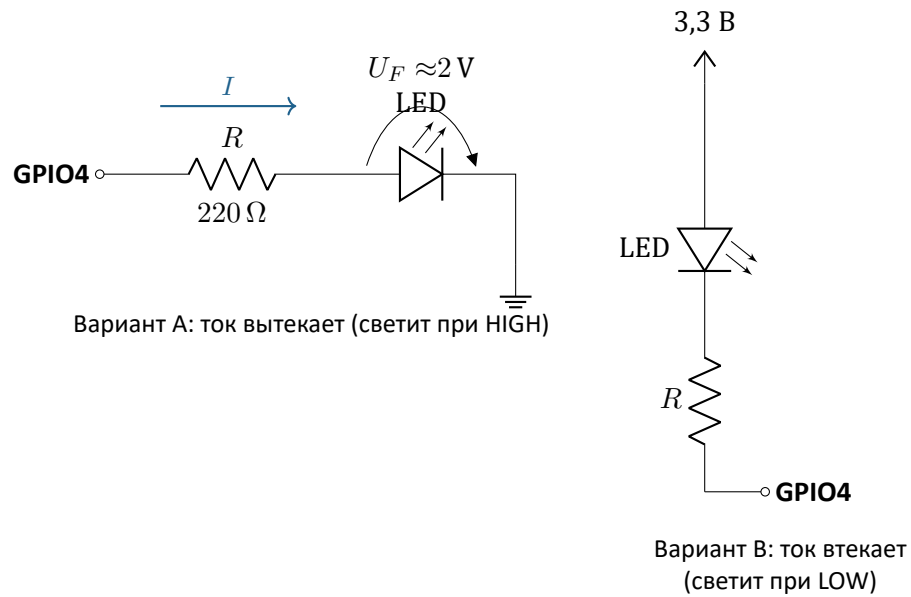


Рис. 7: Два способа подключить светодиод. В проекте используем вариант А

Резистор ограничивает ток. По закону Ома:

$$R = \frac{U_{\text{вых}} - U_F}{I} = \frac{3,3 - 2,0}{0,006} \approx 217 \, \Omega \Rightarrow \text{берём } 220 \, \Omega.$$

Цвет светодиода	U_F , В	R для 5 mA	R для 10 mA
Красный	1,8–2,0	270 Ω	150 Ω
Жёлтый/зелёный	2,0–2,2	220 Ω	120 Ω
Синий/белый	2,8–3,1	68 Ω	33 Ω

Таблица 2: Ориентировочные номиналы резисторов при питании 3,3 В

1.3 Классический Blink

Листинг 2: Мигание светодиодом

```

const int LED_PIN = 4;

void setup() {
  pinMode(LED_PIN, OUTPUT);    // настроить вывод как выход
}

void loop() {
  digitalWrite(LED_PIN, HIGH); // 3,3 В -> светодиод горит
  delay(500);
  digitalWrite(LED_PIN, LOW);  // 0 В -> гаснет
  delay(500);
}

```

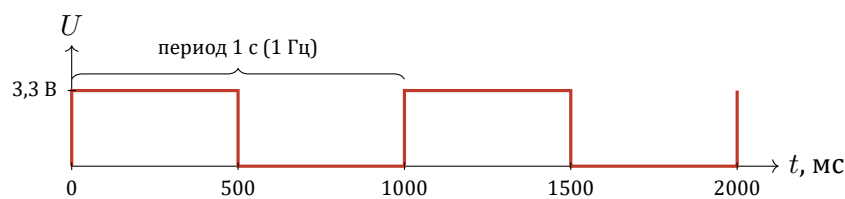


Рис. 8: Напряжение на выводе при работе Blink

1.4 Мигание без delay()

`delay()` останавливает всю программу: пока идёт ожидание, она не может делать ничего другого. В лампе нам предстоит одновременно следить за кнопкой, ручкой, экраном и сетью, поэтому сразу привыкаем к другому приёму — сравнивать текущее время `millis()` с моментом последнего действия:

Листинг 3: Неблокирующее мигание

```

const int LED_PIN = 4;
const uint32_t INTERVAL = 500; // мс
uint32_t lastToggle = 0;
bool ledState = false;

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  uint32_t now = millis();
  if (now - lastToggle >= INTERVAL) { // корректно и при переполнении millis()
    lastToggle = now;
    ledState = !ledState;
    digitalWrite(LED_PIN, ledState);
  }
  // здесь можно делать что-то ещё, не дожидаясь светодиода
}

```

1.5 Встроенный RGB-светодиод

На DevKitC-1 распаян адресный светодиод WS2812 (**GPIO48** или **GPIO38** в зависимости от ревизии). Для него в Arduino-ESP32 3.x есть готовая функция `rgbLedWrite(pin, r, g, b)`:

Листинг 4: Смена цветов встроенного RGB-светодиода

```

const int RGB_PIN = 48;  // на ревизии v1.1 -- 38

void setup() {}

void loop() {
  rgbLedWrite(RGB_PIN, 40, 0, 0); delay(400); // красный (яркость 0..255)
  rgbLedWrite(RGB_PIN, 0, 40, 0); delay(400); // зелёный
  rgbLedWrite(RGB_PIN, 0, 0, 40); delay(400); // синий
  rgbLedWrite(RGB_PIN, 0, 0, 0);  delay(400); // выключен
}

```

1.6 Шаг проекта 1: светодиод лампы и «пульс»

Умная лампа: 1 2 3 4 5 6 7 8 9

LED Serial кнопка ШИМ ручка экран задачи Wi-Fi сон

Подключаем светодиод лампы к **GPIO4** по схеме «вариант А». Встроенный RGB-светодиод будет «пульсом»: мигает зелёным, пока программа работает. Если он замер — программа зависла. Обратите внимание: мигание сделано через `millis()`, поэтому `loop()` свободен для следующих шагов.

Листинг 5: Умная лампа, шаг 1

```

// Умная лампа, шаг 1: светодиод лампы и «пульс» на RGB
const int LAMP_PIN = 4;
const int RGB_PIN  = 48;  // на ревизии v1.1 -- 38

void heartbeat() {  // мигает раз в секунду, не блокируя loop()
  static uint32_t last = 0;
  static bool beat = false;
  if (millis() - last >= 500) {
    last = millis();
    beat = !beat;
    rgbLedWrite(RGB_PIN, 0, beat ? 20 : 0, 0);
  }
}

void setup() {
  pinMode(LAMP_PIN, OUTPUT);
  digitalWrite(LAMP_PIN, HIGH);  // пока просто включаем лампу
}

void loop() {
  heartbeat();
}

```

Совет

Переменные с `static` внутри функции сохраняют значение между вызовами. Так состояние «пульса» не засоряет глобальную область — этот приём пригодится и дальше.

Итоги урока

- `pinMode(pin, OUTPUT)` и `digitalWrite(pin, HIGH/LOW)` — управление выходом.
- Светодиод всегда подключается через резистор; ток вывода ограничен.
- Для отсчёта времени без остановки программы используем `millis()`, а не `delay()`.

Задания для самостоятельной работы

1. Соберите «светофор» из трёх светодиодов с правильной последовательностью и длительностями.
2. Сделайте два светодиода, мигающих с разной частотой (1 Hz и 3 Hz) без `delay()`.
3. Сделайте «пульс» лампы двойным ударом, как сердцебиение: две короткие вспышки и пауза.

Урок 2. Последовательный порт: разговор с компьютером

Цель	понять, как устроен UART, научиться выводить отладочные сообщения и принимать команды с компьютера
Опирается на	урок 1: неблокирующий цикл, светодиод лампы
Шаг проекта	лампа включается и выключается командами <code>on / off</code> из Serial Monitor
Понадобится	то же, что в уроке 1; для последнего раздела — второе устройство с UART (по желанию)

Прежде чем добавлять в лампу кнопки и датчики, нужен способ видеть, что происходит внутри программы. Этим способом будет последовательный порт: через него мы будем проверять результат в каждом следующем уроке.

2.1 Кадр UART

UART — асинхронный протокол: у приёмника и передатчика нет общего тактового сигнала, поэтому обе стороны должны заранее договориться о скорости (бодах). Линия в покое находится в состоянии HIGH.



Рис. 9: Передача символа «А» в формате 8N1 (8 бит данных, без контроля чётности, 1 стоп-бит)

2.2 Serial и Serial Monitor

У ESP32-S3 три аппаратных UART, а в Arduino им соответствуют объекты `Serial`:

Объект Arduino	Контроллер	Выводы по умолчанию	Назначение
Serial	USB CDC или UART0	USB / 43, 44	связь с компьютером
Serial0	UART0	TX 43, RX 44	разъём «UART» на плате
Serial1	UART1	задаются в begin()	свободен
Serial2	UART2	задаются в begin()	свободен

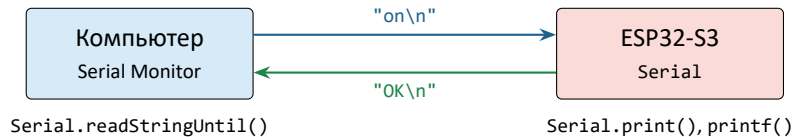


Рис. 10: Обмен строками между компьютером и платой

Основные функции:

- `Serial.begin(115200)` — открыть порт; скорость в Serial Monitor должна совпадать.
- `Serial.print()`, `println()`, `printf()` — отправить текст.
- `Serial.available()` — сколько байт пришло и ждёт чтения.
- `Serial.readStringUntil('\n')` — прочитать строку до символа перевода строки.

Совет

В Serial Monitor выберите окончание строки New Line: тогда каждая отправленная команда будет заканчиваться символом `\n`, и `readStringUntil()` сразу поймёт, где конец команды.

2.3 Шаг проекта 2: команды с компьютера



Добавляем к шагу 1 обработку команд. Функция `handleSerial()` ничего не ждёт: если данных нет, она сразу возвращается, и `loop()` продолжает крутить «пульс».

Листинг 6: Умная лампа, шаг 2 (полный скетч)

```

// Умная лампа, шаг 2: управление из Serial Monitor
const int LAMP_PIN = 4;
const int RGB_PIN = 48; // на ревизии v1.1 -- 38

bool lampOn = false;

void setLamp(bool on) {
  lampOn = on;
  digitalWrite(LAMP_PIN, lampOn);
}

void handleSerial() {
  if (!Serial.available()) return; // команд нет -- не ждём
  String cmd = Serial.readStringUntil('\n');
  cmd.trim(); // убрать \r и пробелы
  if (cmd == "on") { setLamp(true); Serial.println("OK"); }
  else if (cmd == "off") { setLamp(false); Serial.println("OK"); }
  else if (cmd == "status") Serial.printf("lamp=%s, uptime=%lu c\n",
                                          lampOn ? "on" : "off", millis() / 1000);
}
  
```

```

    else                Serial.println("Команды: on, off, status");
}

void heartbeat() {      // из шага 1
    static uint32_t last = 0;
    static bool beat = false;
    if (millis() - last >= 500) {
        last = millis();
        beat = !beat;
        rgbLedWrite(RGB_PIN, 0, beat ? 20 : 0, 0);
    }
}

void setup() {
    Serial.begin(115200);
    pinMode(LAMP_PIN, OUTPUT);
    setLamp(false);
}

void loop() {
    handleSerial();
    heartbeat();
}

```

2.4 Второй UART: связь с другими устройствами

Тот же протокол используют GPS-модули, модемы и другие микроконтроллеры. Благодаря матрице GPIO UART1 и UART2 можно вывести почти на любые выводы.

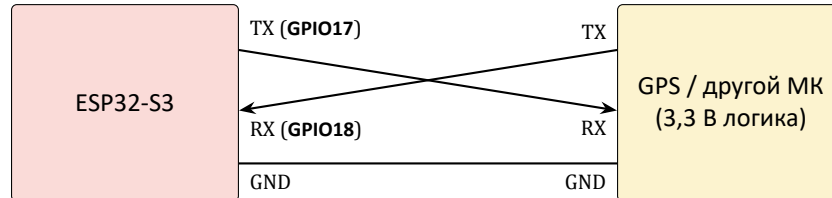


Рис. 11: TX одного устройства соединяется с RX другого; земли объединяются

Листинг 7: Мост: всё, что пришло с модуля, пересылается на компьютер, и наоборот

```

void setup() {
    Serial.begin(115200);
    Serial1.begin(9600, SERIAL_8N1, 18, 17);
}

void loop() {
    while (Serial1.available()) Serial.write(Serial1.read());
    while (Serial.available()) Serial1.write(Serial.read());
}

```

Осторожно

Модули с 5-вольтовой логикой (часть Arduino, некоторые GPS) нельзя подключать к RX ESP32-S3 напрямую — используйте делитель или преобразователь уровней.

Итоги урока

- UART передаёт байты кадрами: старт-бит, 8 бит данных, стоп-бит; скорости сторон должны совпадать.
- Serial — главный инструмент отладки: будем пользоваться им во всех следующих уроках.
- Команды читаем неблокирующе: проверяем `available()` и только тогда читаем строку.

Задания для самостоятельной работы

1. Добавьте в лампу команду `blink N`: мигнуть лампой `N` раз (без `delay()`, по образцу урока 1).
2. Выводите раз в 10 секунд строку с временем работы, не мешая приёму команд.
3. Соедините две платы по UART и передавайте команды лампе с другой платы.

Урок 3. Цифровой вход: кнопки и прерывания

Цель	научиться читать состояние кнопки, понять роль подтягивающих резисторов, побороть дребезг, познакомиться с прерываниями
Опирается на	урок 1 (выходы, <code>millis()</code>), урок 2 (вывод в Serial для проверки)
Шаг проекта	кнопка на GPIO5 включает и выключает лампу
Понадобится	тактовая кнопка

3.1 Проблема «висящего» входа

Вход GPIO имеет очень высокое сопротивление. Если к нему подключить только кнопку на землю, то в отпущенном состоянии вывод никуда не подключён и ловит наводки — читается случайно 0 или 1. Решение — **подтягивающий резистор**, задающий уровень по умолчанию.

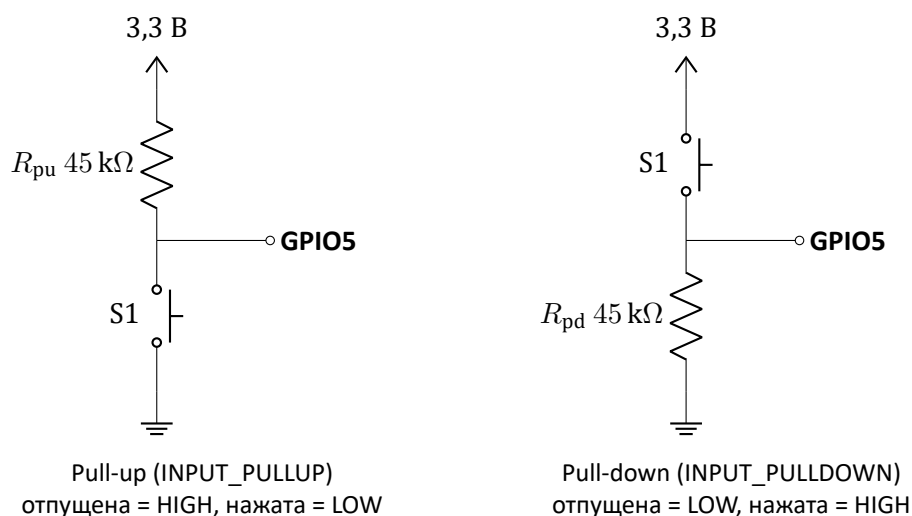


Рис. 12: Подтяжка к питанию и к земле. У ESP32-S3 оба резистора встроены ($\approx 45\text{ k}\Omega$). В проекте используем pull-up: кнопка замыкает GPIO5 на землю

Совет

Встроенная подтяжка слабая. Для длинных проводов или в условиях помех добавьте внешний резистор 4,7 – 10 кΩ.

3.2 Опрос кнопки**Листинг 8:** Состояние кнопки в Serial Monitor

```
const int BTN_PIN = 5;

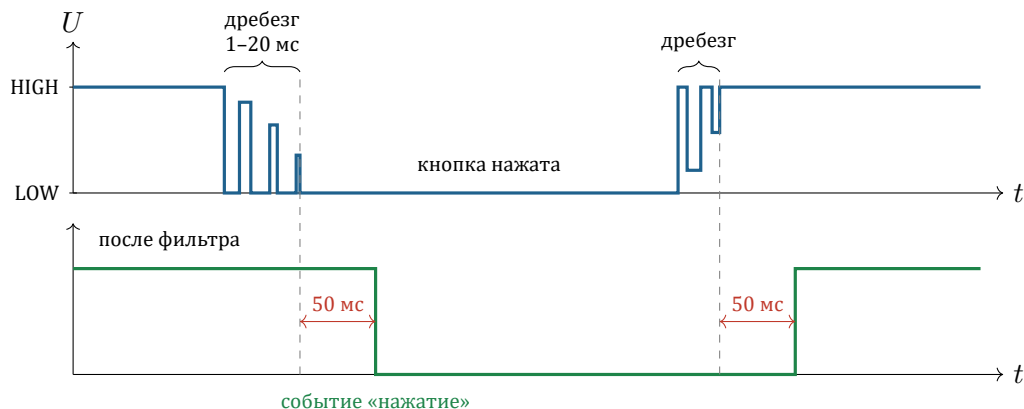
void setup() {
  Serial.begin(115200);
  pinMode(BTN_PIN, INPUT_PULLUP); // включаем встроенную подтяжку
}

void loop() {
  bool pressed = (digitalRead(BTN_PIN) == LOW); // pull-up: нажатие = LOW
  Serial.println(pressed ? "нажата" : "отпущена");
  delay(200);
}
```

Если просто написать «при LOW переключить лампу», лампа будет мигать всё время, пока кнопка удерживается, и срабатывать по нескольку раз на одно нажатие. Нам нужно ловить именно *момент нажатия* — и при этом учестьдребезг.

3.3 Дребезг контактов

Механические контакты при замыкании несколько раз «подпрыгивают». Микроконтроллер, опрашивающий вход миллионы раз в секунду, видит это как серию нажатий.

**Рис. 13:** Дребезг контактов (сверху) и сигнал после программного фильтра (снизу)

Программный фильтр принимает новый уровень, только если он стабилен дольше 50 ms. Оформим его в функцию, которая возвращает true ровно один раз на каждое нажатие:

Листинг 9: Функция buttonPressed(): нажатие с подавлением дребезга

```

const int BTN_PIN = 5;
const uint32_t DEBOUNCE_MS = 50;

bool buttonPressed() {
    static bool stable = digitalRead(BTN_PIN); // подтверждённое состояние
    static bool last = stable;                 // последнее прочитанное
    static uint32_t changed = 0;               // когда уровень менялся
    bool r = digitalRead(BTN_PIN);
    if (r != last) {                           // уровень дёрнулся --
        last = r;                             // перезапускаем отсчёт
        changed = millis();
    }
    if (millis() - changed > DEBOUNCE_MS && r != stable) {
        stable = r;                           // уровень устоялся
        return stable == LOW;                 // событие -- только нажатие
    }
    return false;
}

void setup() {
    Serial.begin(115200);
    pinMode(BTN_PIN, INPUT_PULLUP);
}

void loop() {
    if (buttonPressed()) Serial.println("Нажатие!");
}

```

3.4 Шаг проекта 3: кнопка-выключатель



Подключаем кнопку между **GPIO5** и GND. Функции setLamp(), handleSerial() и heartbeat() остаются из шага 2, buttonPressed() — из раздела выше. Меняются только setup() и loop():

Листинг 10: Умная лампа, шаг 3 (изменения относительно шага 2)

```

// Умная лампа, шаг 3: кнопка-выключатель
const int BTN_PIN = 5;
const uint32_t DEBOUNCE_MS = 50;

// setLamp(), handleSerial(), heartbeat() -- без изменений (шаг 2)
// buttonPressed() -- из раздела 3.3

void setup() {
    Serial.begin(115200);
    pinMode(LAMP_PIN, OUTPUT);
    pinMode(BTN_PIN, INPUT_PULLUP);
    setLamp(false);
}

void loop() {
    handleSerial();
    if (buttonPressed()) setLamp(!lampOn); // нажатие переключает лампу
}

```

```
heartbeat();
}
```

Теперь лампой можно управлять и кнопкой, и командами — оба способа работают одновременно, потому что ни одна функция в `loop()` не останавливает программу.

3.5 Прерывания

Опрос в `loop()` работает, пока цикл крутится быстро. Если программа занята, короткое нажатие можно пропустить. Альтернатива — попросить процессор *самого* вызвать функцию в момент изменения уровня.

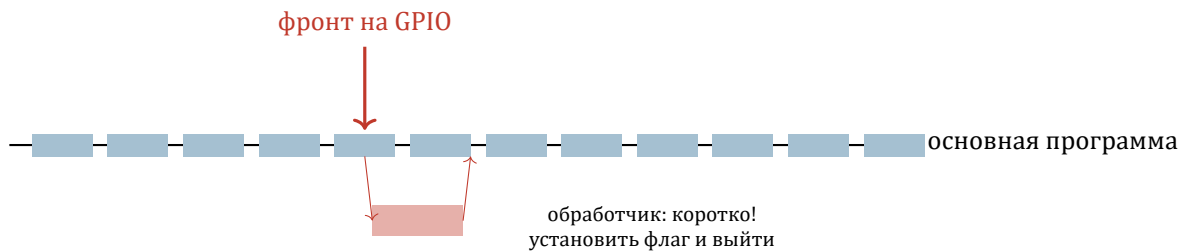


Рис. 14: Прерывание приостанавливает основную программу на время обработчика

Листинг 11: Подсчёт нажатий через прерывание

```
const int BTN_PIN = 5;

volatile uint32_t pressCount = 0; // volatile: меняется в обработчике
volatile uint32_t lastIsrTime = 0;

void IRAM_ATTR onButton() { // IRAM_ATTR: код в быстрой памяти
    uint32_t now = millis();
    if (now - lastIsrTime > 50) { // грубое подавление дребезга
        pressCount++;
        lastIsrTime = now;
    }
}

void setup() {
    Serial.begin(115200);
    pinMode(BTN_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(BTN_PIN), onButton, FALLING);
}

void loop() {
    static uint32_t shown = 0;
    if (pressCount != shown) { // печатаем здесь, а не в обработчике
        shown = pressCount;
        Serial.printf("Нажатий: %lu\n", shown);
    }
}
```

Осторожно

Внутри обработчика прерывания (ISR) нельзя вызывать `delay()`, `Serial.print()` и другие медленные/блокирующие функции. Правило: изменить переменную — и выйти. Всю работу делает основная программа.

Для лампы опроса достаточно, поэтому в проекте оставляем `buttonPressed()`. К прерываниям вернёмся в уроке 7, когда научимся будить из них задачи.

3.6 Strapping-выводы

Уровни на **GPIO0**, **GPIO3**, **GPIO45**, **GPIO46** считываются в момент сброса и определяют режим загрузки и напряжение Flash. Не вешайте на них кнопки или нагрузку, которые могут изменить уровень при старте. Именно поэтому кнопка лампы стоит на **GPIO5**.

Итоги урока

- Вход без подтяжки «висит»; используем `INPUT_PULLUP` и кнопку на землю.
- Нажатие нужно ловить как *событие* и фильтроватьдребезг (20 – 50 ms).
- Прерывание реагирует мгновенно, но обработчик должен быть минимальным.

Задания для самостоятельной работы

1. Используйте кнопку BOOT (**GPIO0**) как вторую кнопку лампы после загрузки.
2. Различайте короткое (< 0,5 с) и длинное (> 1 с) нажатие: длинное печатает в Serial время работы лампы.
3. Замените кнопку сенсорным входом: `touchRead(2)` растёт при касании вывода **GPIO2**.

Урок 4. ШИМ (LEDC): плавная яркость

Цель	понять принцип широтно-импульсной модуляции и научиться задавать «промежуточные» уровни
Опирается на	урок 1 (светодиод), урок 2 (команды), урок 3 (кнопка)
Шаг проекта	у лампы появляется яркость 0–100 %, включение и выключение становятся плавными по восприятию
Понадобится	то же, что раньше; для раздела про серво — сервопривод SG90 (по желанию)

4.1 Что такое ШИМ

`digitalWrite()` умеет только «полностью включено» и «полностью выключено». ESP32-S3 не имеет ЦАП, но может «имитировать» промежуточное напряжение, быстро включая и выключая вывод. Доля времени во включённом состоянии называется **коэффициентом заполнения** (duty cycle):

$$D = \frac{t_{\text{вкл}}}{T}, \quad U_{\text{ср}} = D \cdot 3,3 \text{ V}.$$

При частоте в тысячи герц глаз не замечает мигания и видит среднюю яркость.

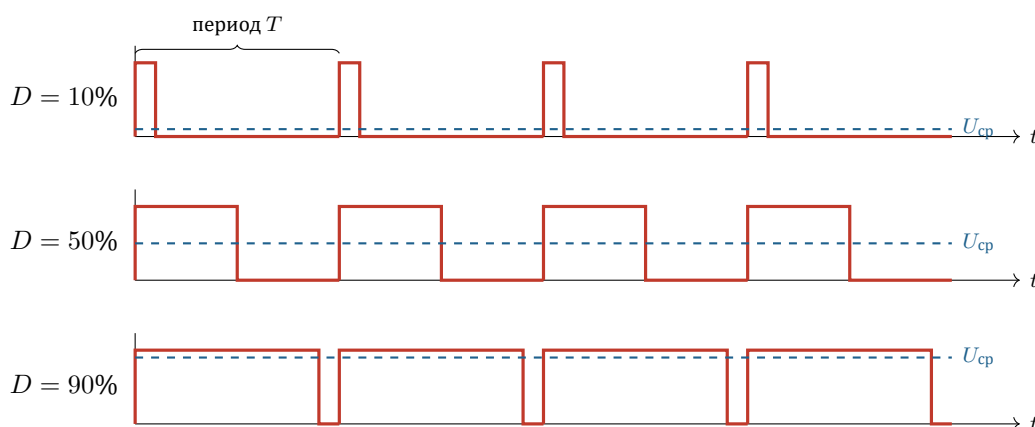


Рис. 15: ШИМ с разным коэффициентом заполнения

4.2 Периферия LEDC

В ESP32-S3 ШИМ формирует модуль **LEDC**: 8 каналов, 4 таймера, разрешение до 14 бит. Частота и разрешение связаны: чем выше частота, тем меньше доступное разрешение ($f_{\text{max}} \approx 80 \text{ MHz}/2^{\text{bits}}$).

Применение	Частота	Разрешение
Светодиоды	5 kHz	8–13 бит
Сервопривод	50 Hz	14 бит
Двигатель через драйвер	10 – 20 kHz	10 бит
Звук (пищалка)	100 – 5000 Hz	8 бит

В Arduino-ESP32 3-х канал выбирается автоматически, а обращаются к нему по номеру вывода: `ledcAttach(pin, freq, bits)` заменяет `pinMode()`, `ledcWrite(pin, duty)` — `digitalWrite()`.

Листинг 12: Плавное «дыхание» светодиода

```

const int LED_PIN = 4;
const uint32_t FREQ = 5000;    // Гц
const uint8_t RES = 8;         // бит -> значения 0..255

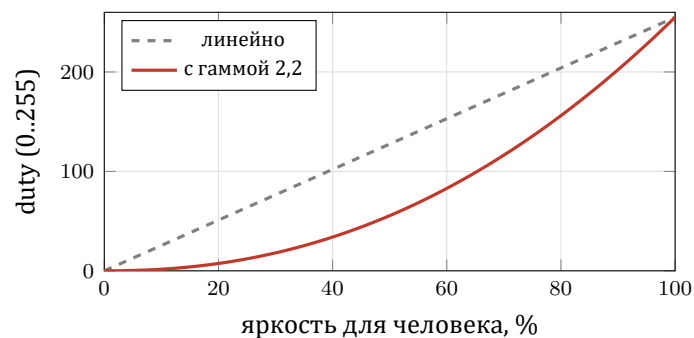
void setup() {
    ledcAttach(LED_PIN, FREQ, RES);
}

void loop() {
    for (int d = 0; d <= 255; d++) { ledcWrite(LED_PIN, d); delay(4); }
    for (int d = 255; d >= 0; d--) { ledcWrite(LED_PIN, d); delay(4); }
}

```

4.3 Яркость и восприятие: гамма-коррекция

Запустив пример, вы заметите: светодиод быстро становится ярким, а дальше почти не меняется. Глаз воспринимает яркость нелинейно, поэтому «50 % для человека» соответствует гораздо меньшему коэффициенту заполнения. Исправляет это гамма-коррекция: $D = D_{\max} \cdot (x/100)^{2,2}$.

**Рис. 16:** Перевод «человеческой» яркости в коэффициент заполнения**4.4 Шаг проекта 4: яркость лампы**

Состояние лампы теперь — это не только «вкл/выкл», но и яркость. Соберём его в структуру, а все изменения будем делать через три функции. Функция `applyLamp()` — единственное место, где состояние превращается в сигнал на выводе.

Листинг 13: Умная лампа, шаг 4 (изменения относительно шага 3)

```

// Умная лампа, шаг 4: плавная яркость через ШИМ
struct LampState {
    bool on;
    int level; // яркость 0..100 %
};
LampState lamp = {false, 50}; // заменяет bool LampOn

uint32_t levelToDuty(int level) { // гамма-коррекция
    return (uint32_t)(255 * pow(level / 100.0, 2.2) + 0.5);
}

```

```

void applyLamp() {                                     // состояние -> вывод
  ledcWrite(LAMP_PIN, lamp.on ? levelToDuty(lamp.level) : 0);
}

void lampSetOn(bool on)      { lamp.on = on;           applyLamp(); }
void lampToggle()           { lamp.on = !lamp.on;      applyLamp(); }
void lampSetLevel(int level) { lamp.level = constrain(level, 0, 100); applyLamp(); }

void handleSerial() {                                   // + команда Level N
  if (!Serial.available()) return;
  String cmd = Serial.readStringUntil('\n');
  cmd.trim();
  if (cmd == "on")          { lampSetOn(true); Serial.println("OK"); }
  else if (cmd == "off")    { lampSetOn(false); Serial.println("OK"); }
  else if (cmd.startsWith("level ")) { lampSetLevel(cmd.substring(6).toInt()); }
  Serial.println("OK"); }
  else if (cmd == "status")  Serial.printf("lamp=%s level=%d%%\n",
                                          lamp.on ? "on" : "off", lamp.level);
  else                      Serial.println("Команды: on, off, level N, status");
}

// heartbeat(), buttonPressed() -- без изменений

void setup() {
  Serial.begin(115200);
  ledcAttach(LAMP_PIN, 5000, 8);           // вместо pinMode(LAMP_PIN, OUTPUT)
  pinMode(BTN_PIN, INPUT_PULLUP);
  applyLamp();
}

void loop() {
  handleSerial();
  if (buttonPressed()) lampToggle();
  heartbeat();
}

```

Проверьте в Serial Monitor: on, затем level 10, level 50, level 100 — ступени яркости должны казаться примерно равными.

4.5 Другое применение ШИМ: сервопривод

Этот раздел не входит в проект, но показывает, что тот же LEDC умеет больше, чем яркость. Хобби-сервопривод ожидает импульсы с периодом 20 ms (50 Hz); длительность импульса задаёт угол.

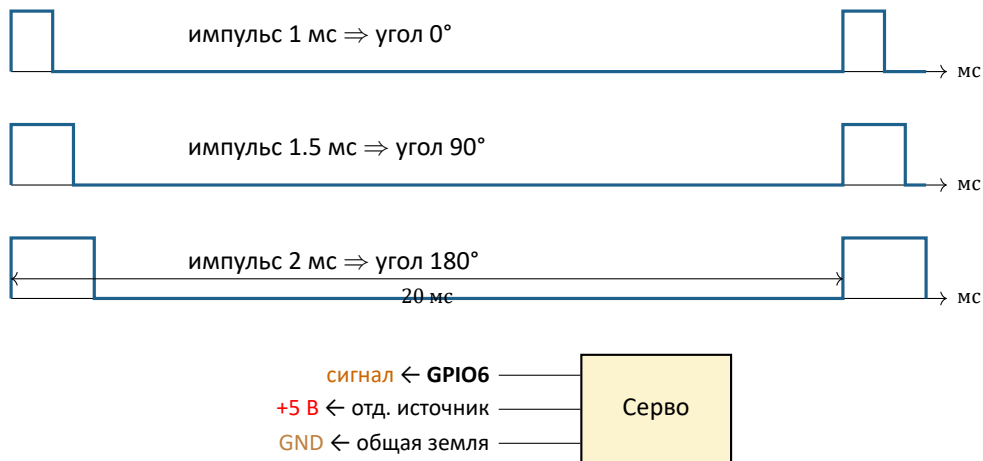


Рис. 17: Управляющие импульсы и подключение сервопривода

Листинг 14: Управление сервоприводом через LEDC

```
const int SERVO_PIN = 6;
const uint8_t RES = 14; // 0..16383 на период 20 мс

uint32_t usToDuty(uint32_t us) { // микросекунды -> значение duty
    return (uint32_t)((uint64_t)us * ((1 << RES) - 1) / 20000);
}

void servoWrite(int angle) { // 0..180 градусов
    uint32_t us = map(angle, 0, 180, 1000, 2000); // у многих серво 500..2500
    ledcWrite(SERVO_PIN, usToDuty(us));
}

void setup() {
    ledcAttach(SERVO_PIN, 50, RES);
}

void loop() {
    for (int a = 0; a <= 180; a += 2) { servoWrite(a); delay(20); }
    for (int a = 180; a >= 0; a -= 2) { servoWrite(a); delay(20); }
}
```

Осторожно

Не питайте сервопривод от вывода 3V3 платы: пусковые токи достигают сотен миллиампер и вызывают перезагрузку ESP32. Используйте отдельный источник 5 V, объединив земли.

Итоги урока

- ШИМ задаёт среднее напряжение долей времени включения; на ESP32-S3 его формирует LEDC.
- `ledcAttach()` + `ledcWrite()` вместо `pinMode()` + `digitalWrite()`.
- Для света используем гамма-коррекцию, иначе регулировка кажется неравномерной.
- Состояние лампы собрано в структуру и меняется только через функции — это пригодится в уроке 7.

Задания для самостоятельной работы

1. Сделайте включение лампы плавным: `ledcFade(pin, from, to, ms)` делает переход аппаратно.
2. Добавьте короткий «щелчок» пьезопищалки при нажатии кнопки: `ledcWriteTone(pin, 2000)` на 30 ms.
3. Сделайте RGB-лампу на трёх каналах LEDC с командой `color R G B`.

Урок 5. АЦП: читаем аналоговые сигналы

Цель	научиться измерять напряжение, понять ограничения АЦП ESP32-S3, измерять напряжения выше 3,3 В
Опирается на	урок 2 (вывод результатов в Serial), урок 4 (яркость лампы)
Шаг проекта	потенциометр становится ручкой яркости
Понадобится	потенциометр 10 kΩ; для раздела про делитель — резисторы 100 kΩ и 33 kΩ

5.1 Как работает АЦП

АЦП последовательного приближения (SAR) преобразует напряжение в число. При разрешении 12 бит диапазон разбивается на $2^{12} = 4096$ уровней:

$$N = \left\lfloor \frac{U_{\text{вх}}}{U_{\text{полн}}} \cdot 4095 \right\rfloor.$$

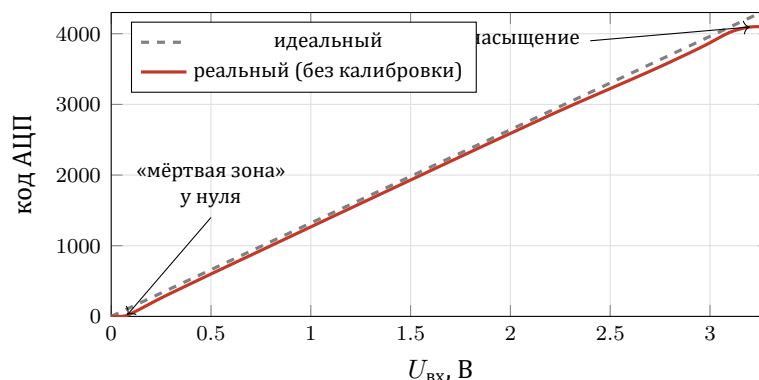


Рис. 18: Характеристика АЦП (иллюстрация). Края диапазона нелинейны — используйте калиброванное чтение

Особенности АЦП ESP32-S3

- Два блока: **ADC1 (GPIO1–GPIO10)** и **ADC2 (GPIO11–GPIO20)**.
- **ADC2 недоступен при работающем Wi-Fi.** Лампа в уроке 8 будет подключена к сети, поэтому ручку ставим на ADC1 — **GPIO1**.
- Внутреннее опорное напряжение $\approx 1,1$ В, поэтому на входе стоит *аттенюатор*. По умолчанию в Arduino выбран 11 dB — рабочий диапазон примерно 0 – 3,1 В.

- Каждый чип откалиброван на заводе (eFuse). Функция `analogReadMilliVolts()` учитывает калибровку и возвращает милливольты — предпочитайте её «сырому» `analogRead()`.

5.2 Потенциометр

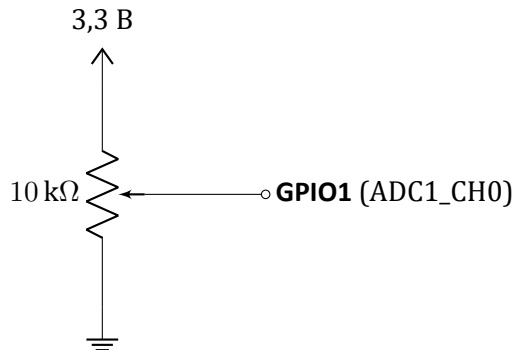


Рис. 19: Потенциометр как регулируемый делитель напряжения

Листинг 15: Чтение потенциометра

```
const int POT_PIN = 1;

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);           // 0..4095
}

void loop() {
  int raw = analogRead(POT_PIN);
  int mv = analogReadMilliVolts(POT_PIN); // с учётом калибровки
  Serial.printf("raw:%d mV:%d\n", raw, mv); // формат для Serial Plotter
  delay(100);
}
```

Совет

Откройте Tools → Serial Plotter: пары «имя:значение» IDE нарисует графиками. Покрутите ручку и обратите внимание на шум — соседние отсчёты отличаются на единицы-десятки милливольт.

5.3 Шаг проекта 5: ручка яркости



Переводим милливольты в проценты яркости и боремся с шумом двумя приёмами:

- **усреднение** — 8 измерений подряд;
- **порог** — яркость меняется, только если ручку повернули больше чем на 3 %. Заодно это позволяет задавать яркость командой `level N`: пока ручку не трогают, команда не будет перезаписана.

Листинг 16: Умная лампа, шаг 5 (изменения относительно шага 4)

```
// Умная лампа, шаг 5: ручка яркости
const int POT_PIN = 1; // ADC1 -- работает вместе с Wi-Fi

int readPotLevel() { // 0..100 %
    uint32_t sum = 0;
    for (int i = 0; i < 8; i++) sum += analogReadMilliVolts(POT_PIN);
    return constrain(map(sum / 8, 0, 3100, 0, 100), 0, 100);
}

int lastPot = -100; // «далеко» -> первое чтение применится

void loop() {
    handleSerial();
    if (buttonPressed()) lampToggle();
    int pot = readPotLevel();
    if (abs(pot - lastPot) >= 3) { // реагируем, только если ручку повернули
        lastPot = pot;
        lampSetLevel(pot);
    }
    heartbeat();
}
```

5.4 Измерение напряжения выше 3,3 В

Чтобы измерить, например, батарею 12 В, понижаем напряжение делителем:

$$U_{\text{АЦП}} = U_{\text{вх}} \cdot \frac{R_2}{R_1 + R_2} = 12 \cdot \frac{33}{100 + 33} \approx 2,98 \text{ В}.$$

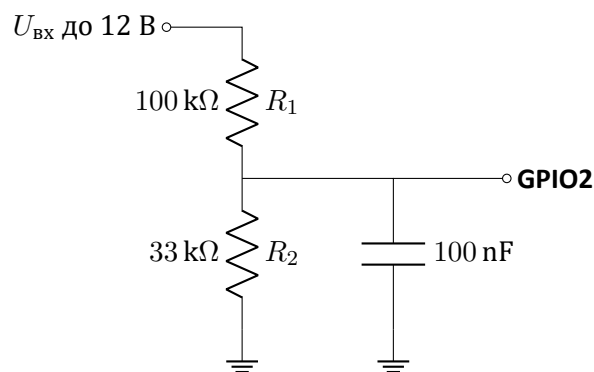


Рис. 20: Делитель напряжения с фильтрующим конденсатором

Листинг 17: Вольтметр с усреднением

```
const int VIN_PIN = 2;
const float K = (100.0 + 33.0) / 33.0;    // коэффициент делителя

float readVoltage() {
    uint32_t sum = 0;
    for (int i = 0; i < 32; i++) sum += analogReadMilliVolts(VIN_PIN);
    return (sum / 32.0) / 1000.0 * K;      // вольты на входе делителя
}

void setup() { Serial.begin(115200); }

void loop() {
    Serial.printf("U = %.2f В\n", readVoltage());
    delay(500);
}
```

Итоги урока

- `analogReadMilliVolts()` даёт калиброванный результат; диапазон — примерно 0–3,1 V.
- ADC2 не работает вместе с Wi-Fi — для сетевых проектов берите ADC1 (**GPIO1–GPIO10**).
- Шум убираем усреднением, а дрожание управления — порогом (гистерезисом).

Задания для самостоятельной работы

1. Добавьте фоторезистор и режим «авто»: лампа ярче, когда в комнате темнее (с гистерезисом!).
2. Сравните показания мультиметра и программы в 5 точках. Постройте график ошибки.
3. Измерьте время одного вызова `readPotLevel()` с помощью `micros()`.

Урок 6. Шина I²C: дисплей

Цель	разобраться в устройстве шины I ² C, найти устройства сканером, вывести информацию на OLED
Опирается на	урок 2 (Serial для сканера), уроки 4–5 (состояние лампы, которое будем показывать)
Шаг проекта	экран показывает, включена ли лампа и её яркость
Понадобится	OLED-дисплей SSD1306 128×64 с интерфейсом I ² C

6.1 Топология шины

I²C использует всего два провода: **SDA** (данные) и **SCL** (такт). Выходы устройств — с *открытым стоком*: они могут только притянуть линию к земле, а в HIGH её возвращают подтягивающие резисторы (тот же принцип, что у кнопки в уроке 3). Каждое ведомое устройство имеет 7-битный адрес.

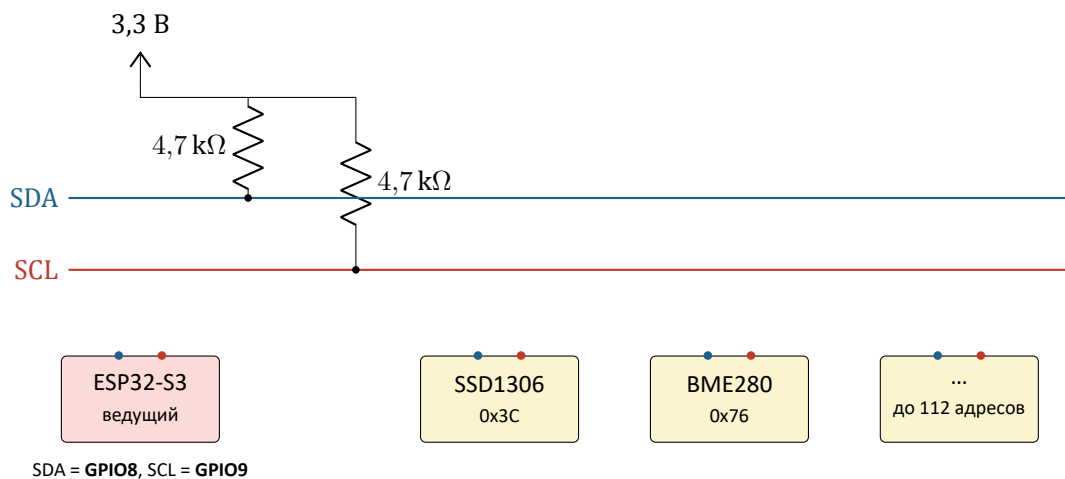


Рис. 21: Шина I²C: один ведущий, несколько ведомых на общих линиях

Совет

Большинство модулей (дисплеи, датчики на платах-переходниках) уже имеют подтягивающие резисторы. Если на шине много модулей, суммарное сопротивление подтяжки может оказаться слишком малым — иногда лишние резисторы приходится выпаивать.

6.2 Временная диаграмма

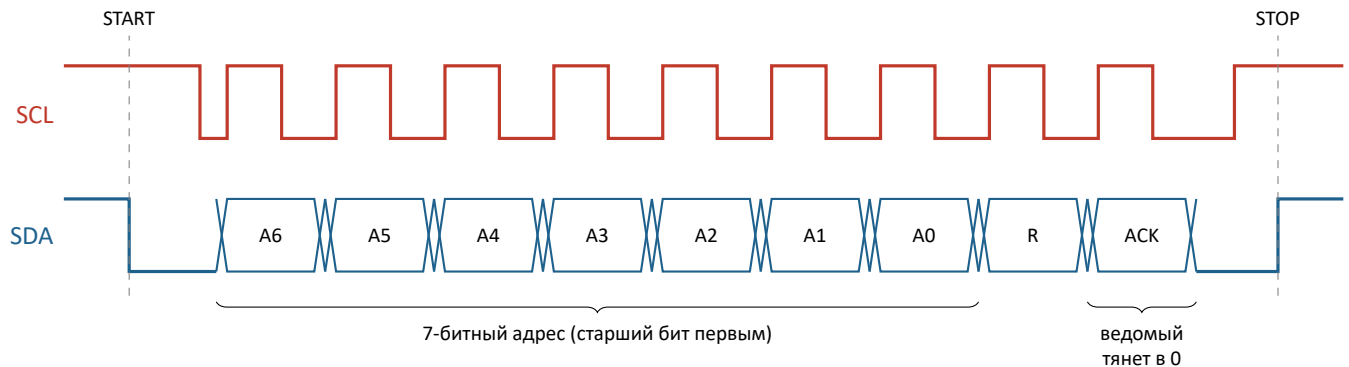


Рис. 22: Начало транзакции: START, адрес, бит чтения/записи, подтверждение ACK, STOP. Данные меняются, пока SCL = LOW, и считываются, пока SCL = HIGH

6.3 Сканер шины

Подключите дисплей: SDA → **GPIO8**, SCL → **GPIO9**, питание 3V3 и GND. Первое, что стоит сделать с новым модулем, — убедиться, что он отвечает, и узнать его адрес.

Листинг 18: Сканер I²C

```
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  delay(500);
  Wire.begin(8, 9);           // SDA, SCL
  Serial.println("Сканирование...");
  int found = 0;
  for (uint8_t addr = 0x08; addr < 0x78; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) { // 0 = устройство ответило ACK
      Serial.printf("  найдено устройство: 0x%02X\n", addr);
      found++;
    }
  }
  Serial.printf("Всего: %d\n", found);
}

void loop() {}
```

6.4 Первый вывод на экран

Установите через Library Manager библиотеки **Adafruit SSD1306** и **Adafruit GFX**. Библиотека рисует в буфере в памяти, а `display()` отправляет весь буфер (1 КБ) по шине.

Листинг 19: Hello на OLED-дисплее

```

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1); // -1: без вывода сброса

void setup() {
  Serial.begin(115200);
  Wire.begin(8, 9);
  Wire.setClock(400000); // Fast mode, 400 кГц
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("Дисплей не найден");
    while (true) delay(1000);
  }
  display.setTextColor(SSD1306_WHITE);
}

void loop() {
  display.clearDisplay(); // очистить буфер
  display.setTextSize(2);
  display.setCursor(0, 0);
  display.print("ESP32-S3");
  display.setTextSize(1);
  display.setCursor(0, 30);
  display.printf("uptime: %lu s", millis() / 1000);
  display.display(); // отправить буфер на экран
  delay(200);
}

```

Совет

Стандартные шрифты Adafruit GFX не содержат кириллицы — для русского текста нужны сторонние шрифты или библиотека U8g2 с кириллическими наборами. В проекте обойдёмся латиницей.

6.5 Шаг проекта 6: экран состояния

Умная лампа: 1 LED 2 Serial 3 кнопка 4 ШИМ 5 ручка 6 экран 7 задачи 8 Wi-Fi 9 сон

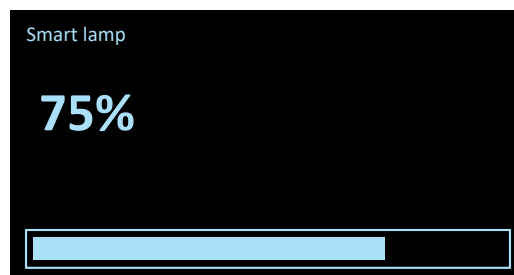


Рис. 23: Экран лампы: заголовок, яркость крупно и полоска-индикатор

Рисовать экран в каждом проходе `loop()` не нужно: отправка буфера занимает десятки миллисекунд. Обновляем его 10 раз в секунду — тем же приёмом с `millis()`, что и «пульс».

Листинг 20: Умная лампа, шаг 6 (изменения относительно шага 5)

```
// Умная лампа, шаг 6: экран состояния
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);

void drawDisplay(const LampState& s) {
  display.clearDisplay();
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.print("Smart lamp");
  display.setTextSize(3);
  display.setCursor(0, 18);
  if (s.on) display.printf("%3d%", s.level);
  else      display.print("OFF");
  display.drawRect(0, 54, 128, 10, SSD1306_WHITE); // рамка полосы
  if (s.on) display.fillRect(2, 56, s.level * 124 / 100, 6, SSD1306_WHITE);
  display.display();
}

void setup() {
  Serial.begin(115200);
  ledcAttach(LAMP_PIN, 5000, 8);
  pinMode(BTN_PIN, INPUT_PULLUP);
  applyLamp();
  Wire.begin(8, 9);
  Wire.setClock(400000);
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) Serial.println("Дисплей не найден");
  display.setTextColor(SSD1306_WHITE);
}

void loop() {
  handleSerial();
  if (buttonPressed()) lampToggle();
  int pot = readPotLevel();
  if (abs(pot - lastPot) >= 3) { lastPot = pot; lampSetLevel(pot); }
  heartbeat();

  static uint32_t lastDraw = 0; // экран -- 10 раз в секунду
  if (millis() - lastDraw >= 100) {
    lastDraw = millis();
    drawDisplay(lamp);
  }
}
```

Лампа готова как автономное устройство. Но посмотрите на `loop()`: в нём уже пять разных дел, и каждое вынуждено «уступать» остальным. Пока отправляется буфер экрана, кнопка не опрашивается; добавь сюда ещё и сеть — и разобраться, кто кому мешает, станет трудно. Эту проблему решает следующий урок.

Итоги урока

- I²C — два провода, подтяжки, адреса; первым делом запускайте сканер.
- Библиотека дисплея рисует в буфере; `display()` отправляет его целиком — это небыстро.
- Медленные действия выполняем периодически, а не на каждом проходе `loop()`.

Задания для самостоятельной работы

1. Добавьте датчик BME280/АНТ20 на ту же шину и показывайте температуру в заголовке экрана.
2. Нарисуйте график изменения яркости за последние 128 обновлений экрана.
3. Сделайте «заставку»: через 10 с без действий экран тускнеет (`display.dim(true)`).

Урок 7. FreeRTOS: задачи и синхронизация

Цель	разделить программу на независимые задачи, организовать безопасный обмен данными, задействовать оба ядра
Опирается на	весь проект шагов 1–6; прерывания из урока 3
Шаг проекта	лампа разбита на задачи: ввод, экран, связь
Понадобится	ничего нового из «железа»

7.1 Зачем нужна операционная система реального времени

В конце урока 6 `loop()` совмещал пять дел, и каждое новое дело замедляло остальные. FreeRTOS позволяет написать каждое дело как отдельную «программу» — **задачу** — со своим бесконечным циклом и приоритетом. Планировщик переключает задачи каждую миллисекунду, а на ESP32-S3 задачи могут выполняться на двух ядрах по-настоящему параллельно. Вы уже пользовались FreeRTOS, не зная об этом: `setup()` и `loop()` выполняются внутри задачи `loopTask` на ядре 1, а `delay()` — это вызов FreeRTOS `vTaskDelay()`, который отдаёт процессор другим задачам.

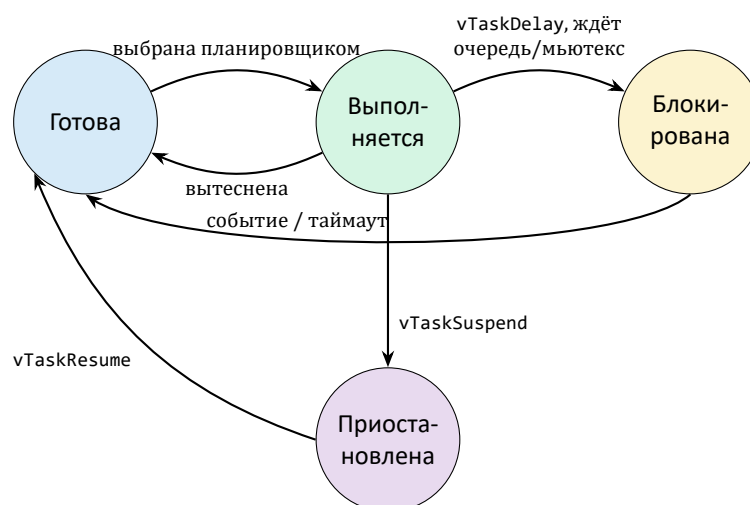


Рис. 24: Состояния задачи FreeRTOS. Задача, ждущая в `vTaskDelay()`, не тратит процессорное время

7.2 Первые задачи

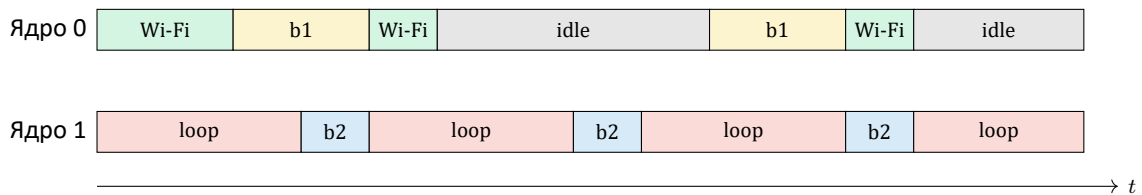


Рис. 25: Задачи на двух ядрах: пока одна ждёт, процессор занят другой

Листинг 21: Две независимые задачи (отдельный эксперимент, светодиоды на GPIO4 и GPIO7)

```
void blinkTask(void* param) {
    int pin = (int)(intptr_t)param;           // номер вывода передан параметром
    int period = (pin == 4) ? 300 : 700;
    pinMode(pin, OUTPUT);
    for (;;) {                                // задача никогда не завершается сама
        digitalWrite(pin, !digitalRead(pin));
        vTaskDelay(pdMS_TO_TICKS(period));    // отдаём процессор другим задачам
    }
}

void setup() {
    Serial.begin(115200);
    //          функция   имя   стек   параметр   приор.   дескр.   ядро
    xTaskCreatePinnedToCore(blinkTask, "b1", 2048, (void*)(intptr_t)4, 1, NULL, 0);
    xTaskCreatePinnedToCore(blinkTask, "b2", 2048, (void*)(intptr_t)7, 1, NULL, 1);
}

void loop() {
    Serial.printf("loop() на ядре %d\n", xPortGetCoreID());
    delay(2000);
}
```

Сравните с уроком 1: там для двух частот мигания пришлось вести отдельные таймеры на `millis()`, а здесь каждая задача просто «спит» столько, сколько ей нужно.

7.3 Общие данные: очередь и мьютекс

Когда несколько задач работают с одними данными, возникает опасность: одна задача может прочитать структуру в тот момент, когда другая записала её лишь наполовину. Для передачи данных между задачами используется **очередь** — потокобезопасный FIFO-буфер.



Рис. 26: Очередь развязывает задачи по времени

Листинг 22: Производитель и потребитель

```

QueueHandle_t q;

void sensorTask(void*) {
    for (;;) {
        int mv = analogReadMilliVolts(1);
        xQueueSend(q, &mv, 0);           // не ждать, если очередь полна
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void printTask(void*) {
    int mv;
    for (;;) {
        if (xQueueReceive(q, &mv, portMAX_DELAY) == pdTRUE) { // спим до прихода данных
            Serial.printf("U = %d мВ\n", mv);
        }
    }
}

void setup() {
    Serial.begin(115200);
    q = xQueueCreate(5, sizeof(int));
    xTaskCreatePinnedToCore(sensorTask, "sensor", 3072, NULL, 2, NULL, 1);
    xTaskCreatePinnedToCore(printTask, "print", 4096, NULL, 1, NULL, 1);
}

void loop() { vTaskDelete(NULL); } // LoopTask больше не нужен

```

Если же данные — это общее *состояние*, которое все читают и меняют (как lamp), удобнее **мьютекс**: «ключ», который задача берёт перед работой с данными и возвращает после. Пока ключ у одной задачи, остальные ждут.

Примитив	Когда использовать
Очередь (xQueue*)	передача данных между задачами или из прерывания (xQueueSendFromISR)
Мьютекс (xSemaphoreCreateMutex) Двоичный семафор	защита общего ресурса: переменных, Serial, шины I ² C сигнал «событие произошло» (например, из прерывания в задаче)
Уведомления (xTaskNotify)	самый быстрый и лёгкий сигнал конкретной задаче
Программный таймер (xTimerCreate)	периодический вызов короткой функции без отдельной задачи

Таблица 3: Основные средства синхронизации FreeRTOS

7.4 Шаг проекта 7: лампа на задачах



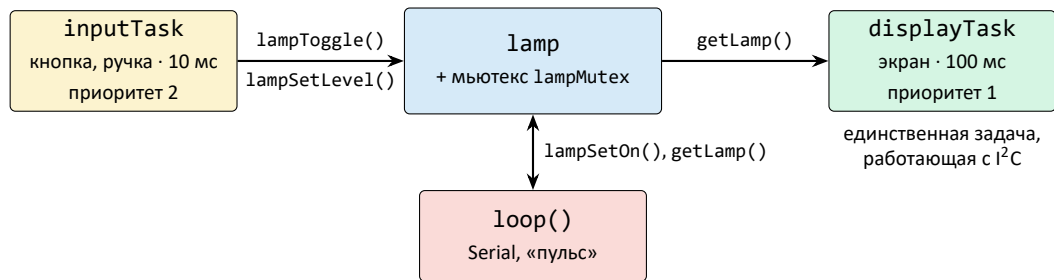


Рис. 27: Архитектура лампы: задачи обращаются к общему состоянию только через функции под мьютексом

Здесь окупается решение из урока 4: состояние лампы меняется только через функции, поэтому защитить его достаточно в одном месте — внутри этих функций.

Листинг 23: Умная лампа, шаг 7 (изменения относительно шага 6)

```

// Умная лампа, шаг 7: задачи FreeRTOS
SemaphoreHandle_t lampMutex;           // «ключ» к состоянию Lamp
TaskHandle_t displayHandle;           // дескриптор задачи экрана

LampState getLamp() {                  // безопасная копия состояния
    xSemaphoreTake(lampMutex, portMAX_DELAY);
    LampState copy = lamp;
    xSemaphoreGive(lampMutex);
    return copy;
}

void lampSetOn(bool on) {
    xSemaphoreTake(lampMutex, portMAX_DELAY);
    lamp.on = on;
    applyLamp();
    xSemaphoreGive(lampMutex);
}

// lampToggle() и lampSetLevel() -- по тому же образцу;
// в handleSerial() для команды status читаем getLamp() вместо lamp

void inputTask(void*) {                // кнопка и ручка: каждые 10 мс
    int lastPot = -100;
    for (;;) {
        if (buttonPressed()) lampToggle();
        int pot = readPotLevel();
        if (abs(pot - lastPot) >= 3) { lastPot = pot; lampSetLevel(pot); }
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

void displayTask(void*) {               // экран: 10 раз в секунду
    for (;;) {
        drawDisplay(getLamp());
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void setup() {
    Serial.begin(115200);
    lampMutex = xSemaphoreCreateMutex(); // до запуска задач!
    ledcAttach(LAMP_PIN, 5000, 8);
    pinMode(BTN_PIN, INPUT_PULLUP);
}
  
```

```

    applyLamp();
    Wire.begin(8, 9);
    Wire.setClock(400000);
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) Serial.println("Дисплей не найден");
    display.setTextColor(SSD1306_WHITE);
    xTaskCreatePinnedToCore(inputTask, "input", 4096, NULL, 2, NULL, 1);
    xTaskCreatePinnedToCore(displayTask, "display", 4096, NULL, 1, &displayHandle, 1);
}

void loop() {                                     // остались только Serial и «пульс»
    handleSerial();
    heartbeat();
    delay(10);
}

```

Кнопка теперь опрашивается каждые 10 ms независимо от того, чем занят экран: у задачи ввода выше приоритет, и планировщик прервёт рисование, как только ей пора работать.

Осторожно

Переполнение стека — частая причина перезагрузок с сообщением `Stack canary watchpoint triggered`. Проверяйте запас функцией `uxTaskGetStackHighWaterMark(NULL)` и увеличивайте размер стека при использовании `printf`, `String` и больших локальных массивов.

Итоги урока

- Задача — функция с бесконечным циклом; `vTaskDelay()` отдаёт процессор другим.
- Данные между задачами передаём очередью, общее состояние защищаем мьютексом.
- Каждое периферийное устройство (экран, Serial) лучше закрепить за одной задачей.

Задания для самостоятельной работы

1. Переведите кнопку на прерывание (урок 3): обработчик будит `inputTask` через `vTaskNotifyGiveFromISR()`.
2. Раз в 5 секунд печатайте в Serial запас стека каждой задачи.
3. Вынесите «пульс» в отдельную задачу и уберите `delay(10)` из `loop()`. Что изменилось?

Урок 8. Wi-Fi и веб-сервер

Цель	подключить ESP32-S3 к Wi-Fi и сделать веб-страницу для управления лампой
Опирается на	урок 7 (функции лампы под мьютексом), урок 5 (почему ручка на ADC1)
Шаг проекта	лампой можно управлять с телефона: кнопка вкл/выкл и ползунок яркости
Понадобится	Wi-Fi-сеть на 2,4 GHz

8.1 Режимы Wi-Fi

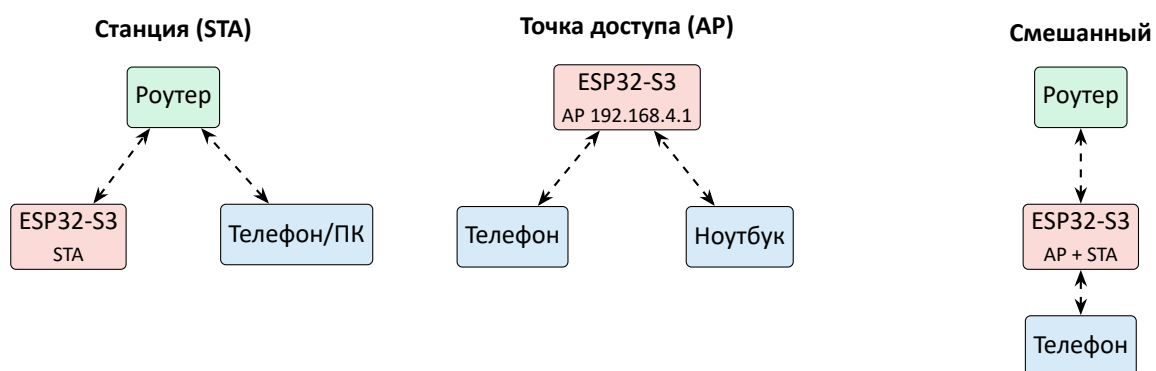


Рис. 28: Три режима работы Wi-Fi. Лампа будет работать как станция

Стек Wi-Fi — это несколько задач FreeRTOS, которые по умолчанию работают на ядре 0. Наши задачи закреплены за ядром 1, поэтому сеть не мешает кнопке и экрану.

8.2 Подключение к сети

Листинг 24: Подключение к Wi-Fi

```

#include <WiFi.h>

const char* SSID = "MyNetwork";
const char* PASS = "MyPassword";

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(SSID, PASS);
  Serial.print("Подключение");
  uint32_t start = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - start < 15000) {
    delay(300);
    Serial.print(".");
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.printf("\nIP: %s, RSSI: %d дБм\n",
                  WiFi.localIP().toString().c_str(), WiFi.RSSI());
  } else {
    Serial.println("\nНе удалось подключиться");
  }
}
  
```

```

}
}
void loop() {}

```

Осторожно

ESP32-S3 работает только в диапазоне 2,4 GHz. Сети 5 GHz он не увидит.

8.3 Как работает HTTP-запрос

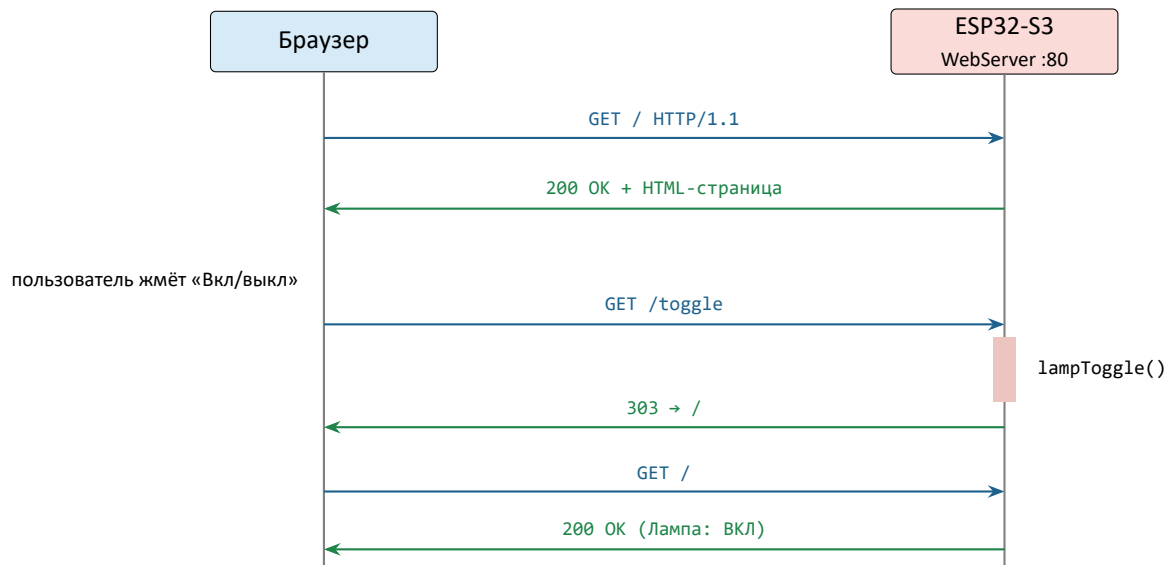


Рис. 29: Последовательность запросов при нажатии кнопки на веб-странице

Каждой ссылке на странице соответствует обработчик на плате. После действия сервер отвечает перенаправлением (303) на главную страницу — браузер сразу загружает её с новым состоянием.

8.4 Шаг проекта 8: управление через браузер



Страница строится из текущего состояния (`getLamp()`), а обработчики вызывают те же функции `lampToggle()` и `lampSetLevel()`, что кнопка и ручка. Благодаря мьютексу из урока 7 не важно, из какой задачи пришла команда. Подключение к сети не ждём: лампа работает и без Wi-Fi.

Листинг 25: Умная лампа, шаг 8 (изменения относительно шага 7)

```

// Умная лампа, шаг 8: управление через браузер
#include <WiFi.h>
#include <WebServer.h>

const char* SSID = "MyNetwork";
const char* PASS = "MyPassword";
WebServer server(80);

String page() {

```

```

LampState s = getLamp();
String html = "<!DOCTYPE html><html><head><meta charset='utf-8'>"
              "<meta name='viewport' content='width=device-width'>"
              "<title>Лампа</title></head><body style='font-family:sans-serif'>";

html += "<h1>Лампа: ";
html += s.on ? "ВКЛ" : "ВЫКЛ";
html += "</h1><p><a href='/toggle'><button>Вкл/выкл</button></a></p>"
        "<form action='/level'>Яркость: <input type='range' name='v' "
        "min='0' max='100' value='" + String(s.level) + "' "
        "onchange='this.form.submit()'></form></body></html>";
return html;
}

void redirectHome() {
  server.sendHeader("Location", "/");
  server.send(303);
}

void setupWeb() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(SSID, PASS); // не ждём: лампа работает и без сети
  server.on("/", []() { server.send(200, "text/html; charset=utf-8", page()); });
  server.on("/toggle", []() { lampToggle(); redirectHome(); });
  server.on("/level", []() { lampSetLevel(server.arg("v").toInt()); redirectHome(); });
  server.begin();
}

// setup(): в конце добавьте вызов setupWeb()
// drawDisplay(): в заголовке показываем адрес, когда сеть подключена:
// display.print(WiFi.isConnected() ? WiFi.localIP().toString() : String("Smart Lamp"));

void loop() {
  handleSerial();
  server.handleClient(); // обработать входящие запросы
  heartbeat();
  delay(2);
}

```

После загрузки IP-адрес появится в заголовке экрана. Откройте его в браузере телефона, подключённого к той же сети, и попробуйте одновременно крутить ручку и двигать ползунок.

8.5 Если роутера нет: собственная точка доступа

ESP32-S3 может сам создать сеть. Замените в `setupWeb()` две строки подключения на:

```

WiFi.softAP("SmartLamp", "12345678"); // пароль не короче 8 символов
Serial.println(WiFi.softAPIP());      // обычно 192.168.4.1

```

Совет

Чтобы не запоминать IP-адрес, подключите mDNS: `#include <ESPmDNS.h>` и `MDNS.begin("lamp");` — страница откроется по адресу `http://lamp.local/`.

Итоги урока

- Режимы Wi-Fi: станция, точка доступа, смешанный; только 2,4 GHz.
- `WebServer`: адрес → обработчик; `handleClient()` вызываем постоянно.
- Сеть — ещё один источник команд; лампа принимает их через те же функции, что и кнопка.

Задания для самостоятельной работы

1. Сделайте адрес `/api/state`, возвращающий JSON: `{"on":true,"level":75}`.
2. Если за 15 секунд не удалось подключиться к роутеру, переходите в режим точки доступа.
3. Добавьте на страницу автообновление раз в 2 секунды, чтобы видеть изменения от ручки.

Урок 9. Сон и энергосбережение

Цель	разобраться в режимах сна ESP32-S3 и научиться будить чип таймером и кнопкой
Опирается на	урок 3 (кнопка), урок 7 (управление задачами), урок 8 (Wi-Fi как главный потребитель)
Шаг проекта	выключенная лампа через минуту засыпает и просыпается от кнопки
Понадобится	мультиметр для измерения тока (по желанию)

9.1 Режимы питания

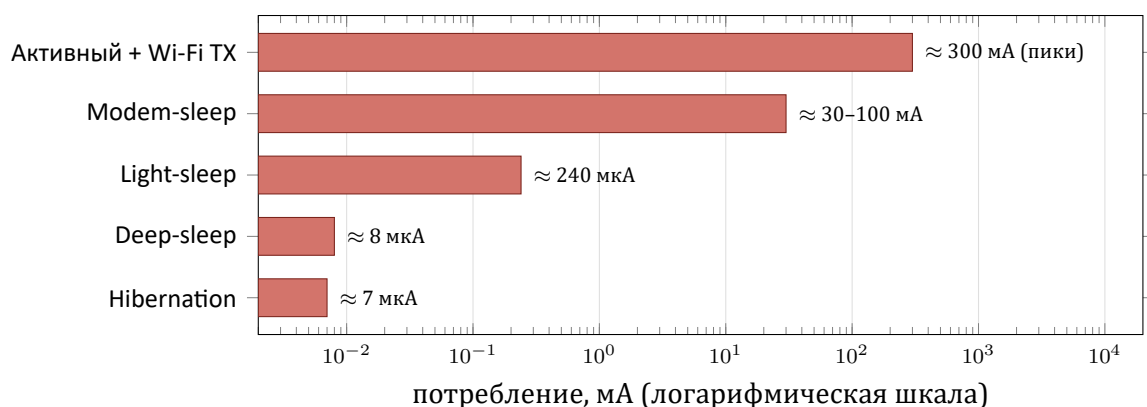


Рис. 30: Типичное потребление самого чипа в разных режимах (ориентировочно, по datasheet). Отладочная плата со стабилизатором и светодиодом питания потребляет заметно больше

Режим	Что работает	Что происходит при пробуждении
Modem-sleep	CPU, радио выключается между пакетами	— (обычная работа)
Light-sleep	всё заморожено, память сохранена	программа продолжается со следующей строки
Deep-sleep	только RTC, RTC-память и ULP	полная перезагрузка , <code>setup()</code> заново
Hibernation	только RTC-таймер и некоторые выводы	полная перезагрузка

Главное про deep-sleep: при пробуждении программа начинается с начала, а обычные переменные теряются. Сохранить данные можно в RTC-памяти — переменные с атрибутом `RTC_DATA_ATTR`.

9.2 Сколько проработает устройство от батареи

Типичный батарейный датчик просыпается, делает работу и снова засыпает:

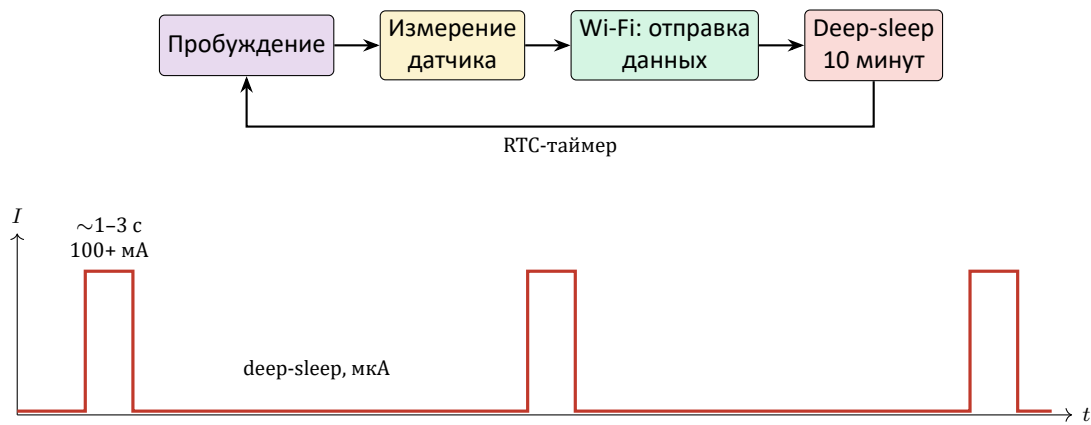


Рис. 31: Устройство почти всё время спит; средний ток определяется длительностью активной фазы

$$I_{\text{cp}} = \frac{I_{\text{акт}} t_{\text{акт}} + I_{\text{сон}} t_{\text{сон}}}{t_{\text{акт}} + t_{\text{сон}}} = \frac{120 \text{ mA} \cdot 2 \text{ s} + 0,01 \text{ mA} \cdot 598 \text{ s}}{600 \text{ s}} \approx 0,41 \text{ mA}.$$

Аккумулятор 2000 mAh прослужит $\approx 2000/0,41 \approx 4900$ часов, то есть около полугода.

9.3 Deep-sleep по таймеру

Листинг 26: Пробуждение по таймеру с сохранением счётчика

```
#include "esp_sleep.h"

RTC_DATA_ATTR int bootCount = 0;           // переживает deep-sleep

const uint64_t SLEEP_US = 10ULL * 1000000; // 10 секунд

void setup() {
    Serial.begin(115200);
    delay(500);
    bootCount++;
    Serial.printf("Пробуждение №%d, причина: %d\n",
                  bootCount, esp_sleep_get_wakeup_cause());

    // ... здесь измерения и отправка данных ...

    esp_sleep_enable_timer_wakeup(SLEEP_US);
    Serial.println("Засыпаю");
    Serial.flush();
    esp_deep_sleep_start();                 // сюда программа уже не вернётся
}

void loop() {}                             // не вызывается
```

Осторожно

При питании через нативный USB-порт после засыпания USB-устройство исчезает из системы, и Serial Monitor «отваливается». Это нормально. Если плата не даёт залить новую прошивку, войдите в загрузчик вручную (BOOT + RST, урок 0).

9.4 Пробуждение кнопкой

Проснуться можно по уровню на RTC-выводе (**GPIO0–GPIO21**) — кнопка лампы на **GPIO5** под-ходит. Во сне обычная подтяжка из урока 3 отключается, поэтому включаем подтяжку RTC-домена:

```
#include "driver/rtc_io.h"           // функции rtc_gpio_*

rtc_gpio_pullup_en(GPIO_NUM_5);     // подтяжка, работающая во сне
rtc_gpio_pulldown_dis(GPIO_NUM_5);
esp_sleep_enable_ext0_wakeup(GPIO_NUM_5, 0); // проснуться при LOW (кнопка нажата)
esp_deep_sleep_start();
```

9.5 Шаг проекта 9: лампа засыпает

Умная лампа: 1 2 3 4 5 6 7 8 9

LED Serial кнопка ШИМ ручка экран задачи Wi-Fi сон

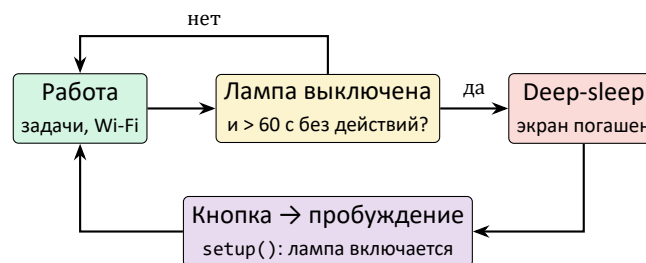


Рис. 32: Цикл жизни лампы с энергосбережением

Что нужно сделать:

- запоминать время последнего действия — удобно прямо в `applyLamp()`, через которую проходят все изменения;
- перед сном остановить задачу экрана (`vTaskSuspend`, урок 7) и погасить дисплей;
- сохранить состояние лампы в RTC-памяти, чтобы после пробуждения вернуть прежнюю яркость;
- при пробуждении кнопкой сразу включить лампу — пользователь ведь нажал кнопку именно для этого.

Листинг 27: Умная лампа, шаг 9 (изменения относительно шага 8)

```
// Умная лампа, шаг 9: сон, когда лампа выключена
#include "driver/rtc_io.h"

const uint32_t SLEEP_AFTER_MS = 60000; // минута без дела -> сон
volatile uint32_t lastActivity = 0;

RTC_DATA_ATTR LampState lamp = {false, 50}; // было: LampState lamp -- теперь переживает сон

void applyLamp() {
    ledcWrite(LAMP_PIN, lamp.on ? levelToDuty(lamp.level) : 0);
    lastActivity = millis(); // любое изменение = активность
}

void goToSleep() {
```

```

vTaskSuspend(displayHandle);           // экран больше не рисуется
display.ssd1306_command(SSD1306_DISPLAYOFF);
rgbledWrite(RGB_PIN, 0, 0, 0);
rtc_gpio_pullup_en((gpio_num_t)BTN_PIN); // подтяжка, работающая во сне
rtc_gpio_pulldown_dis((gpio_num_t)BTN_PIN);
esp_sleep_enable_ext0_wakeup((gpio_num_t)BTN_PIN, 0);
Serial.println("Сплю. Разбудит кнопка.");
Serial.flush();
esp_deep_sleep_start();
}

void setup() {
    rtc_gpio_deinit((gpio_num_t)BTN_PIN); // вернуть кнопку в обычный режим GPIO
    if (esp_sleep_get_wakeup_cause() == ESP_SLEEP_WAKEUP_EXT0)
        lamp.on = true; // разбудили кнопкой -> включаем
    // ... дальше без изменений (шаг 8)
}

void loop() {
    handleSerial();
    server.handleClient();
    heartbeat();
    if (!getLamp().on && millis() - lastActivity > SLEEP_AFTER_MS) goToSleep();
    delay(2);
}

```

Совет

Функция `buttonPressed()` из урока 3 читает начальное состояние кнопки при первом вызове. Поэтому нажатие, разбудившее лампу, не будет воспринято ещё раз как «выключить» — даже если кнопку ещё не отпустили, когда программа запустилась.

Итоги урока

- Deep-sleep снижает ток до микроампер, но пробуждение — это перезапуск программы.
- Нужные данные храним в `RTC_DATA_ATTR`, причину пробуждения узнаём через `esp_sleep_get_wakeup_cause()`.
- Будить можно таймером, уровнем на RTC-выводе, сенсорным входом.

Задания для самостоятельной работы

1. Измерьте ток платы мультиметром в работе и во сне. Отключите светодиод питания и сравните.
2. Добавьте второй источник пробуждения — таймер на 1 час — и при таком пробуждении только печатайте статистику и засыпайте снова, не включая лампу.
3. Считайте в RTC-памяти, сколько раз лампу включали, и показывайте счётчик на экране.

Приложение А. Полный код умной лампы

Итоговый скетч объединяет все девять шагов. Комментарии в заголовках разделов указывают урок, в котором появилась соответствующая часть. Для сборки нужны библиотеки Adafruit SSD1306 и Adafruit GFX; не забудьте указать имя и пароль своей сети.

Листинг 28: Умная лампа: итоговый скетч

```
// Умная лампа на ESP32-S3 -- итоговый скетч курса
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "driver/rtc_io.h"

// ----- Выводы -----
const int LAMP_PIN = 4;           // светодиод лампы           (урок 1)
const int RGB_PIN  = 48;          // встроенный RGB, v1.1: 38
const int BTN_PIN  = 5;           // кнопка на GND             (урок 3)
const int POT_PIN  = 1;           // потенциометр, ADC1        (урок 5)
const int SDA_PIN  = 8, SCL_PIN = 9; // OLED SSD1306              (урок 6)

// ----- Настройки -----
const char* SSID = "MyNetwork";   // (урок 8)
const char* PASS = "MyPassword";
const uint32_t DEBOUNCE_MS = 50; // (урок 3)
const uint32_t SLEEP_AFTER_MS = 60000; // (урок 9)

// ----- Состояние (уроки 4, 7, 9) -----
struct LampState {
    bool on;
    int level; // яркость 0..100 %
};
RTC_DATA_ATTR LampState lamp = {false, 50}; // переживает deep-sleep
SemaphoreHandle_t lampMutex;
TaskHandle_t displayHandle;
volatile uint32_t lastActivity = 0;

Adafruit_SSD1306 display(128, 64, &Wire, -1);
WebServer server(80);

// ----- Лампа (уроки 4, 7) -----
uint32_t levelToDuty(int level) { // гамма-коррекция
    return (uint32_t)(255 * pow(level / 100.0, 2.2) + 0.5);
}

void applyLamp() { // вызывать под мьютексом
    ledcWrite(LAMP_PIN, lamp.on ? levelToDuty(lamp.level) : 0);
    lastActivity = millis();
}

LampState getLamp() {
    xSemaphoreTake(lampMutex, portMAX_DELAY);
    LampState copy = lamp;
    xSemaphoreGive(lampMutex);
    return copy;
}
```

```

void lampSetOn(bool on) {
    xSemaphoreTake(lampMutex, portMAX_DELAY);
    lamp.on = on;
    applyLamp();
    xSemaphoreGive(lampMutex);
}

void lampToggle() {
    xSemaphoreTake(lampMutex, portMAX_DELAY);
    lamp.on = !lamp.on;
    applyLamp();
    xSemaphoreGive(lampMutex);
}

void lampSetLevel(int level) {
    xSemaphoreTake(lampMutex, portMAX_DELAY);
    lamp.level = constrain(level, 0, 100);
    applyLamp();
    xSemaphoreGive(lampMutex);
}

// ----- Ввод (уроки 3, 5) -----
bool buttonPressed() { // true один раз на нажатие
    static bool stable = digitalRead(BTN_PIN);
    static bool last = stable;
    static uint32_t changed = 0;
    bool r = digitalRead(BTN_PIN);
    if (r != last) { last = r; changed = millis(); }
    if (millis() - changed > DEBOUNCE_MS && r != stable) {
        stable = r;
        return stable == LOW;
    }
    return false;
}

int readPotLevel() { // 0..100 %
    uint32_t sum = 0;
    for (int i = 0; i < 8; i++) sum += analogReadMilliVolts(POT_PIN);
    return constrain(map(sum / 8, 0, 3100, 0, 100), 0, 100);
}

// ----- Экран (урок 6) -----
void drawDisplay(const LampState& s) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setCursor(0, 0);
    display.print(WiFi.isConnected() ? WiFi.localIP().toString() : String("Smart lamp"));
    display.setTextSize(3);
    display.setCursor(0, 18);
    if (s.on) display.printf("%3d%", s.level);
    else display.print("OFF");
    display.drawRect(0, 54, 128, 10, SSD1306_WHITE);
    if (s.on) display.fillRect(2, 56, s.level * 124 / 100, 6, SSD1306_WHITE);
    display.display();
}

// ----- Задачи (урок 7) -----
void inputTask(void*) {

```

```

int lastPot = -100;
for (;;) {
    if (buttonPressed()) lampToggle();
    int pot = readPotLevel();
    if (abs(pot - lastPot) >= 3) { lastPot = pot; lampSetLevel(pot); }
    vTaskDelay(pdMS_TO_TICKS(10));
}
}

void displayTask(void*) {
    for (;;) {
        drawDisplay(getLamp());
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

// ----- Serial и «пульс» (уроки 1, 2) -----
void handleSerial() {
    if (!Serial.available()) return;
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();
    if (cmd == "on") { lampSetOn(true); Serial.println("OK"); }
    else if (cmd == "off") { lampSetOn(false); Serial.println("OK"); }
    else if (cmd.startsWith("level ")) { lampSetLevel(cmd.substring(6).toInt()); }
    Serial.println("OK"); }
    else if (cmd == "status") {
        LampState s = getLamp();
        Serial.printf("lamp=%s level=%d%% ip=%s\n", s.on ? "on" : "off", s.level,
            WiFi.localIP().toString().c_str());
    }
    else Serial.println("Команды: on, off, level N, status");
}

void heartbeat() {
    static uint32_t last = 0;
    static bool beat = false;
    if (millis() - last >= 500) {
        last = millis();
        beat = !beat;
        rgbLedWrite(RED_PIN, 0, beat ? 20 : 0, 0);
    }
}

// ----- Веб (урок 8) -----
String page() {
    LampState s = getLamp();
    String html = "<!DOCTYPE html><html><head><meta charset='utf-8'>"
        "<meta name='viewport' content='width=device-width'>"
        "<title>Лампа</title></head><body style='font-family:sans-serif'>";

    html += "<h1>Лампа: ";
    html += s.on ? "ВКЛ" : "ВЫКЛ";
    html += "</h1><p><a href='/toggle'><button>Вкл/выкл</button></a></p>"
        "<form action='/level'>Яркость: <input type='range' name='v' "
        "min='0' max='100' value='" + String(s.level) + "' "
        "onchange='this.form.submit()'></form></body></html>";

    return html;
}

void redirectHome() {

```

```

server.sendHeader("Location", "/");
server.send(303);
}

void setupWeb() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(SSID, PASS);
  server.on("/", []() { server.send(200, "text/html; charset=utf-8", page()); });
  server.on("/toggle", []() { lampToggle(); redirectHome(); });
  server.on("/level", []() { lampSetLevel(server.arg("v").toInt()); redirectHome(); });
  server.begin();
}

// ----- Сон (урок 9) -----
void goToSleep() {
  vTaskSuspend(displayHandle);
  display.ssd1306_command(SSD1306_DISPLAYOFF);
  rgbLedWrite(RGB_PIN, 0, 0, 0);
  rtc_gpio_pullup_en((gpio_num_t)BTN_PIN);
  rtc_gpiopulldown_dis((gpio_num_t)BTN_PIN);
  esp_sleep_enable_ext0_wakeup((gpio_num_t)BTN_PIN, 0);
  Serial.println("Сплю. Разбудит кнопка.");
  Serial.flush();
  esp_deep_sleep_start();
}

// ----- Точка входа -----
void setup() {
  Serial.begin(115200);
  rtc_gpio_deinit((gpio_num_t)BTN_PIN);
  if (esp_sleep_get_wakeup_cause() == ESP_SLEEP_WAKEUP_EXT0) lamp.on = true;

  lampMutex = xSemaphoreCreateMutex();
  ledcAttach(LAMP_PIN, 5000, 8);
  pinMode(BTN_PIN, INPUT_PULLUP);
  applyLamp();

  Wire.begin(SDA_PIN, SCL_PIN);
  Wire.setClock(400000);
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) Serial.println("Дисплей не найден");
  display.setTextColor(SSD1306_WHITE);

  xTaskCreatePinnedToCore(inputTask, "input", 4096, NULL, 2, NULL, 1);
  xTaskCreatePinnedToCore(displayTask, "display", 4096, NULL, 1, &displayHandle, 1);
  setupWeb();
}

void loop() {
  handleSerial();
  server.handleClient();
  heartbeat();
  if (!getLamp().on && millis() - lastActivity > SLEEP_AFTER_MS) goToSleep();
  delay(2);
}

```

Приложение В. Справочник

«Опасные» выводы ESP32-S3

Выводы	Особенность
GPIO0, 3, 45, 46	strapping: влияют на режим загрузки; GPIO0 — кнопка BOOT
GPIO19, 20	нативный USB (D−, D+)
GPIO26–32	SPI-Flash — не использовать
GPIO33–37	заняты при Octal Flash/PSRAM (35–37 — при Octal PSRAM)
GPIO39–42	JTAG (MTCK, MTDO, MTDI, MTMS) — свободны, если не нужна JTAG-отладка по этим выводам
GPIO43, 44	UART0 (TX, RX) — сообщения загрузчика
GPIO11–20	ADC2 — недоступен при работающем Wi-Fi
GPIO22–25	не существуют

Что изучать дальше

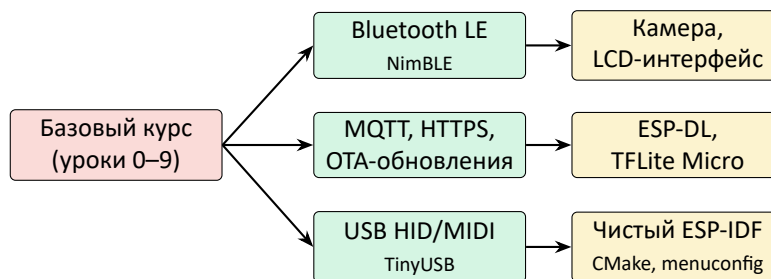


Рис. 33: Возможные направления для дальнейшего изучения. Для лампы естественное продолжение — MQTT (интеграция с умным домом) и OTA (обновление прошивки по воздуху)

Полезные источники

- ESP32-S3 Datasheet и Technical Reference Manual — <https://www.espressif.com/en/support/documents/technical-documents>
- Документация Arduino-ESP32 — <https://docs.espressif.com/projects/arduino-esp32/>
- Руководство ESP-IDF — <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/>
- Описание платы DevKitC-1 — раздел *ESP32-S3-DevKitC-1 User Guide* на <https://docs.espressif.com>